

Java 24 — Simple Source Files and Instance Main Methods (Fourth Preview)

Property	Details
Java Version	24
Year	2025
Feature	Simple Source Files and Instance Main Methods (Fourth Preview)
JEP	JEP 495
UniqueClassName	SimpleSourceFilesInstanceMain
Status in Java 24	Fourth Preview
First preview	Java 21
Second preview	Java 22
Third preview	Java 23
Primary goal	Make it easier for beginners and small programs to write and run Java without ceremony
Main concepts	Simple source files + flexible main methods + implicitly declared classes
Typical beginner code	<code>void main() { println("Hello"); }</code>
Target audience	Beginners, teaching, small scripts, experiments, small utilities
Enterprise value	Limited; conventional classes remain preferable for most production applications

JEP 495 is the fourth preview of the feature previously known as **Implicitly Declared Classes and Instance Main Methods**. It continues the goal of allowing a Java program to be written with substantially less ceremony.

1. What Is the Feature?

Traditionally, the smallest Java program looks like:

```
public class HelloWorld {  
    public static void main(String[] args) {
```

```

    System.out.println("Hello, World!");
}
}

```

There is a lot of ceremony:

```

public class
  ↓
class name
  ↓
public static
  ↓
void main
  ↓
String[] args
  ↓
System.out.println()

```

JEP 495 allows a much simpler source file:

```

void main() {
    System.out.println("Hello, World!");
}

```

The source file itself can act as the program entry point.

Conceptually, Java supplies the surrounding class structure for you.

2. Traditional Java vs Simple Source File

Traditional Java	Simple Source File
Explicit class required	Class can be implicitly declared
Explicit main signature	Simpler main allowed
public static void main(String[] args)	void main() can be used
More boilerplate	Less boilerplate
Good for production	Good for learning/small programs
Explicit class identity	Compiler/runtime supplies class structure

Traditional:

```
public class Calculator {  
    public static void main(String[] args) {  
        System.out.println(10 + 20);  
    }  
}
```

Simple:

```
void main() {  
    System.out.println(10 + 20);  
}
```

The second form focuses on the actual task rather than Java's structural ceremony.

3. Why Was It Introduced?

Java has historically required beginners to learn several concepts simultaneously:

```
Class  
↓  
public  
↓  
static  
↓  
void  
↓  
main  
↓  
String[]  
↓  
System.out
```

For someone learning programming, these aren't necessarily relevant to the first lesson.

The feature allows teaching to begin with:

```
void main() {  
    System.out.println("Hello");  
}
```

and introduce classes, objects, access modifiers, static methods, and other concepts later.

This is part of Java's broader effort to improve the **on-ramp for beginners** while retaining the traditional Java programming model.

4. Simple Source File

A source file can contain a main method without explicitly declaring a class.

Example:

```
void main() {  
    System.out.println("Welcome to Java");  
}
```

The class is **implicitly declared**.

Conceptually, the compiler treats it somewhat like:

```
class Main {  
  
    void main() {  
        System.out.println("Welcome to Java");  
    }  
}
```

The actual rules are more precise than this simplified illustration, but this is a useful mental model.

5. Instance main() Method

One of the major changes is that the entry point does not necessarily need to be:

```
public static void main(String[] args)
```

An instance main method can be:

```
void main() {  
    System.out.println("Hello");  
}
```

The runtime can create an instance and invoke the method.

Conceptually:

```
Program starts  
↓  
Create instance  
↓  
Invoke main()  
↓  
Application executes
```

This removes the need for beginners to understand static just to start a program.

6. Example — Simple Calculator

```
void main() {  
  
    int a = 10;  
    int b = 20;  
  
    System.out.println(a + b);  
}
```

Output:

30

Traditional version:

```
public class Calculator {  
  
    public static void main(String[] args) {  
  
        int a = 10;  
        int b = 20;  
  
        System.out.println(a + b);  
    }  
}
```

Aspect	Traditional	Simple Source
Lines of ceremony	More	Less
Class declaration	Required	Implicit
Static required	Yes for traditional main	No
Beginner friendly	☆☆	☆☆☆☆
Enterprise suitability	☆☆☆☆	☆☆

7. Advantages

Advantage	Explanation
Much less boilerplate	Removes explicit class and traditional main ceremony
Beginner friendly	Students can focus on programming concepts
Simpler first Java program	Hello World becomes very small
Instance main()	Beginners don't need to understand static immediately
Good for teaching	Teachers can introduce concepts incrementally
Good for small programs	Tiny utilities can be concise
Fast experimentation	Useful for trying Java language features
Script-like experience	Makes Java feel less verbose for small tasks
Encourages experimentation	Lower barrier to creating a Java program
Can introduce APIs gradually	Class/object concepts can come later
Preserves Java language underneath	It isn't a completely different programming model
Useful for code snippets	Small examples become easier to read

8. Disadvantages

Disadvantage	Explanation
Preview feature in Java 24	Not yet a permanent language feature
Requires preview support	Java 24 compilation/execution needs preview enabled
Less explicit structure	Class identity is not immediately visible
Not ideal for large applications	Large applications benefit from explicit architecture
Can confuse experienced developers	Multiple valid main forms increase language surface area
Tooling compatibility	Some tools may assume conventional source

	structure
Framework integration can be awkward	Frameworks often expect explicit classes
Package organization is less obvious	Simple source files are best for small programs
Not a performance feature	Doesn't make applications faster
Potential educational transition issue	Students eventually need to learn traditional classes anyway
Can encourage overly simplistic code	Beginners may postpone learning good design practices
Not appropriate for every deployment model	Enterprise packaging often benefits from explicit types

9. When to Use

Situation	Recommendation	Reason
Learning Java	👍 Excellent	Minimal syntax
First programming lesson	👍 Excellent	Focus on logic
Hello World	👍 Excellent	Very concise
Small algorithm experiment	👍 Excellent	Less ceremony
Coding exercise	👍 Excellent	Focus on problem
Small command-line utility	👍 Good	Simple entry point
API experimentation	👍 Good	Fast experimentation
Java language demonstrations	👍 Excellent	Small examples
Temporary developer utility	👍 Good	Less boilerplate
Prototype	👍 Good	Quick to create

Enterprise application	☒ Usually avoid	Explicit structure is valuable
Framework-based application	☒ Usually avoid	Framework conventions matter

10. When NOT to Use

Situation	Why Avoid It?	Better Choice
Large enterprise application	Explicit architecture is clearer	Normal classes
Spring Boot application	Framework expects conventional structures	Explicit classes
Large REST API	Many components/classes are required	Explicit classes
Public library	Public types should be explicit	Normal classes
Shared domain model	Explicit class identity matters	Normal classes
Complex inheritance hierarchy	Implicit class is less suitable	Explicit classes
Multiple entry points	Explicit classes are clearer	Explicit main classes
Complex dependency injection	Framework/container integration	Explicit classes
Team standard requires Java LTS/stable features	Java 24 feature is preview	Traditional syntax
Production system requiring predictable tooling	Preview syntax adds risk	Conventional Java
Large package/module architecture	Explicit boundaries help	Explicit classes

11. Real-Time Use Case — Teaching

A teacher wants to explain variables.

Instead of:

```
public class Variables {  
  
    public static void main(String[] args) {  
  
        int age = 25;  
        double salary = 50000;  
  
        System.out.println(age);  
        System.out.println(salary);  
    }  
}
```

students can start with:

```
void main() {  
  
    int age = 25;  
    double salary = 50000;  
  
    System.out.println(age);  
    System.out.println(salary);  
}
```

The lesson can focus on:

Variable
↓
Data type
↓
Value
↓
Expression
↓
Output

rather than:

Class
↓
public
↓
static
↓
main
↓

String[]

12. Real-Time Use Case — Algorithm Experiment

Suppose you want to test binary search:

```
void main() {  
  
    int[] numbers = {10, 20, 30, 40, 50};  
  
    int target = 30;  
  
    int low = 0;  
    int high = numbers.length - 1;  
  
    while (low <= high) {  
  
        int mid = (low + high) >>> 1;  
  
        if (numbers[mid] == target) {  
            System.out.println("Found");  
            return;  
        }  
  
        if (numbers[mid] < target) {  
            low = mid + 1;  
        } else {  
            high = mid - 1;  
        }  
    }  
  
    System.out.println("Not found");  
}
```

There is no need to create:

```
public class BinarySearchDemo
```

just to test the algorithm.

13. Real-Time Use Case — Small File Utility

A developer needs a quick utility:

```
import java.nio.file.Files;
```

```
import java.nio.file.Path;

void main() throws Exception {

    long lines =
        Files.lines(Path.of("application.log"))
            .count();

    System.out.println("Lines = " + lines);
}
```

This can be convenient for temporary tooling.

For a permanent production utility, however, an explicit class may still be preferable.

14. Real-Time Use Case — API Experimentation

Suppose you want to test an API:

```
import java.time.LocalDate;

void main() {

    LocalDate today = LocalDate.now();

    System.out.println(today);
}
```

This is useful when learning:

- `LocalDate`
- `LocalTime`
- `Collections`
- `Streams`
- `HTTP Client`
- `Regex`
- `Files`
- `JSON libraries`
- `Java language features`

You can concentrate on the API instead of creating a complete application structure.

15. Real-Time Use Case — Coding Practice

For an algorithm problem:

```

void main() {

    int n = 10;

    for (int i = 1; i <= n; i++) {
        System.out.println(i);
    }
}

```

This is particularly useful in:

- Programming classes
- Algorithm practice
- Interview preparation
- Competitive programming experiments
- Java tutorials

16. Real-Time Use Case — Developer Scratch Program

Suppose you want to test how a Java API behaves:

```

void main() {

    var text = "hello";

    System.out.println(text.toUpperCase());
}

```

Instead of creating:

```

ScratchTest.java
↓
class ScratchTest
↓
main()
↓
test

```

you can keep the source focused on the experiment.

17. Simple Source Files vs Traditional Java

Category	Simple Source File	Traditional Class
Boilerplate	★★★★★	★★

Beginner friendliness	★★★★★	★★★
Small programs	★★★★★	★★★
Large programs	★★	★★★★★
Explicit architecture	★★	★★★★★
Framework compatibility	★★	★★★★★
Teaching	★★★★★	★★★★
Prototyping	★★★★★	★★★
Production enterprise	★★	★★★★★
Public API	★	★★★★★

18. Instance Main vs Static Main

Aspect	Instance main()	Traditional static main()
static required	✗	✓
Object required	Runtime can create instance	✗
Beginner friendly	★★★★★	★★★
Conventional Java	★★	★★★★★
Framework familiarity	★★	★★★★★
Small programs	★★★★★	★★★★
Large applications	★★	★★★★★
Existing tooling compatibility	?	?
Preview in Java 24	?	✗

19. Important: It Does Not Remove Classes From Java

This feature should **not** be interpreted as:

"Java no longer needs classes."

That's incorrect.

The underlying Java model still has classes and objects.

The feature simply lets the compiler/runtime provide some of the ceremony automatically for simple programs.

Think of it as:

Simple source
↓
Compiler supplies structural ceremony
↓
Java class
↓
JVM execution

rather than:

Simple source
↓
No class exists

20. Relation to Other Java Features

Feature	Purpose
Simple Source Files	Reduce source-code ceremony
Instance Main Methods	Allow simpler entry-point methods
Implicitly Declared Classes	Avoid explicit class declaration
Unnamed Variables/Patterns	Reduce unnecessary variable names
Local Variable Type Inference (var)	Reduce type verbosity
Text Blocks	Reduce multiline-string boilerplate
Records	Reduce data-carrier boilerplate

Together, these features move Java toward:

Less ceremony



More focus on intent

while retaining Java's type system and object-oriented model.

21. Alternate Features

There is no exact replacement because this feature specifically changes the source/program-entry experience.

Alternative	Purpose	When Better
Traditional public static void main	Standard Java entry point	Production applications
Explicit class + instance main	Explicit object-oriented structure	When using instance state
Static factory methods	Controlled object creation	Complex construction
JShell	Interactive Java experimentation	REPL-style experimentation
Build-tool application plugins	Application packaging/execution	Enterprise builds
JAR executable	Standard Java deployment	Production applications
Framework entry point	Framework-managed startup	Spring/Micronaut/Quarkus etc.

22. Simple Source Files vs JShell

This is an especially useful comparison.

Aspect	Simple Source File	JShell
Persistent source file	✓	Optional
Full Java program	✓	Not primarily

Interactive REPL	✗	✓
Beginner friendly	☆☆☆☆	☆☆☆☆
Quick API experiments	☆☆☆☆	☆☆☆☆
Build into application	✓	✗
Production deployment	Possible	✗
Multiple methods/classes	✓	Limited/different model
Best use	Small programs	Interactive experimentation

Use JShell when you want an interactive console.

Use Simple Source Files when you want an actual Java source program with minimal ceremony.

23. Simple Source Files vs Traditional main

Requirement	Simple Source	Traditional Main
Hello World	?	?
Beginner exercises	?	?
Production application	?	?
Framework application	?/?	?
Large codebase	?	?
Public API	?	?
Scratch program	?	?
Learning APIs	?	?
Preview-free Java	?	?
Maximum tooling compatibility	?	?

24. Performance Considerations

Area	Impact
Application throughput	✗ No inherent improvement
Algorithm performance	✗ No inherent improvement
JVM performance	✗ No meaningful direct benefit
Startup performance	✗ Not the purpose
Memory usage	✗ Not the purpose
Source-code size	☑ Reduced
Developer productivity	☑ Improved for small programs
Learning curve	☑ Reduced
Build process	☑ Preview support required

This is a **developer-experience/language ergonomics feature**, not a performance feature.

25. Best Practices

Practice	Recommendation
Use for learning	☑ Excellent
Use for small examples	☑ Excellent
Use for API experiments	☑ Excellent
Use for temporary utilities	☑ Good
Use for tutorials	☑ Excellent
Use for prototypes	☑ Good

Use for large enterprise applications	☑ Prefer explicit classes
Use for public libraries	☑ Prefer explicit classes
Use with complex inheritance	☑ Prefer explicit classes
Keep production architecture explicit	☑ Recommended
Don't confuse simplicity with good architecture	☑ Important
Remember Java 24 preview status	☑ Important

26. Decision Table

Question	Recommendation
Are you learning Java?	☑ Use it
Teaching programming fundamentals?	☑ Use it
Writing Hello World?	☑ Use it
Testing a Java API?	☑ Use it
Writing a small algorithm?	☑ Use it
Creating a temporary utility?	☑ Use it
Building a large application?	☑ Use explicit classes
Building Spring Boot application?	☑ Use conventional structure
Creating a public library?	☑ Use explicit classes
Need maximum tooling compatibility?	☑ Traditional main
Avoiding preview features?	☑ Traditional main
Need complex object-oriented architecture?	☑ Explicit classes

27. Overall Advantages vs Disadvantages

Category	Assessment
Beginner experience	☑ Excellent
Teaching	☑ Excellent
Small programs	☑ Excellent
API experimentation	☑ Excellent
Coding exercises	☑ Excellent
Temporary utilities	☑ Very good
Prototyping	☑ Good
Large enterprise systems	☑ Limited
Framework applications	☑ Limited
Public libraries	☑ Poor fit
Explicit architecture	☑ Weaker
Performance	☑ No direct benefit
Java syntax simplicity	☑ Excellent
Production maturity in Java 24	☑ Preview

28. Final Recommendation

Scenario	Best Choice
Beginner learning Java	★ Simple Source Files
Hello World/tutorial	★ Simple Source Files
Algorithm practice	★ Simple Source Files
API experimentation	★ Simple Source Files / JShell
Small script-like utility	★ Simple Source Files

Prototype	★ Simple Source Files
Small production utility	🔗 Depends
Spring Boot application	★ Traditional classes
Large enterprise application	★ Traditional classes
Public library	★ Traditional classes
Complex OOP architecture	★ Traditional classes
Stable Java-only project	★ Traditional main
Interactive experimentation	★ JShell

Bottom line

Simple Source Files and Instance Main Methods (Fourth Preview) make Java much easier to start with, but they are not intended to replace conventional Java application architecture.

Traditional:

```
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

can become:

```
void main() {
    System.out.println("Hello World");
}
```

The biggest benefit is therefore:

Less ceremony

↓

Faster learning

↓

Faster experimentation

↓

Simpler small programs

rather than:

Less ceremony



Faster JVM

My recommendation: use this feature heavily for **learning, teaching, demonstrations, coding exercises, API experiments, and small utilities**. For **large enterprise applications, frameworks, reusable libraries, and long-lived production systems**, continue to prefer explicit classes and conventional entry points because they make architecture, ownership, tooling, and dependencies clearer.

One important Java-24 consideration: **JEP 495 is still a preview feature**, so it should be enabled deliberately rather than treated like a finalized Java language feature.

Java 24 — Quantum-Resistant ML-KEM

Property	Details
Java Version	24
Year	2025
Feature	Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM)
JEP	JEP 496
UniqueClassName	QuantumResistantMLKEM
Status	Final
Standard	NIST FIPS 203
Security category	Post-Quantum Cryptography (PQC)
Primary purpose	Securely establish shared symmetric keys over an insecure channel
Java provider	SunJCE
Supported parameter sets	ML-KEM-512, ML-KEM-768, ML-KEM-1024
Main APIs	KeyPairGenerator, KeyFactory, KEM
Java 24 package	javax.crypto / java.security APIs

Preview?	✗ No — final feature
----------	----------------------

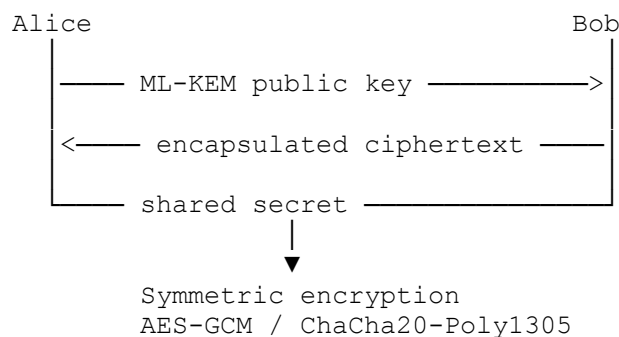
Java 24 added a standard JDK implementation of ML-KEM, standardized by NIST as FIPS 203. Oracle describes ML-KEM as a quantum-resistant KEM used to secure symmetric keys over insecure communication channels. ([Oracle Docs](#))

1. What Is ML-KEM?

ML-KEM = Module-Lattice-Based Key Encapsulation Mechanism.

Its purpose is **not to encrypt an entire application message directly**.

Its primary purpose is to allow two parties to establish a shared secret:



The resulting shared secret can then be used with a symmetric encryption algorithm.

Oracle specifically describes KEMs as mechanisms for securing symmetric keys over insecure communication channels using public-key cryptography. ([Oracle Docs](#))

2. Why Is It Called "Quantum-Resistant"?

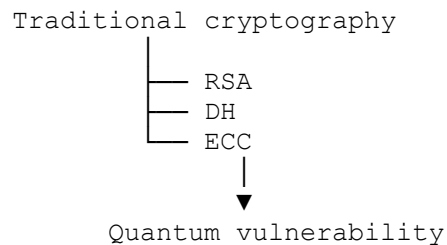
Traditional public-key algorithms such as:

- RSA
- Diffie-Hellman
- Elliptic Curve Diffie-Hellman
- Elliptic Curve cryptography

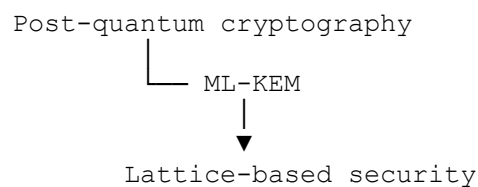
are vulnerable to sufficiently capable quantum computers because of algorithms such as **Shor's algorithm**.

ML-KEM is designed around a different mathematical problem based on lattices.

Conceptually:



versus:



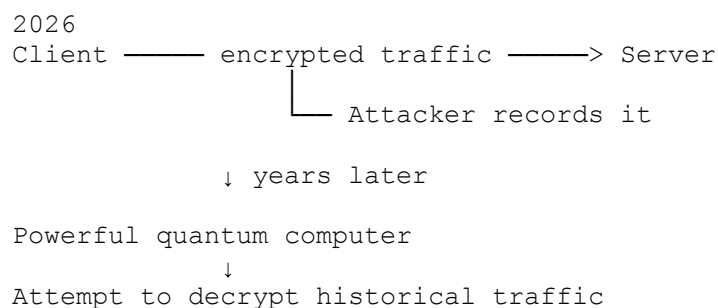
ML-KEM is standardized in **NIST FIPS 203**. ([Java](#))

3. The "Harvest Now, Decrypt Later" Problem

One major reason organizations are migrating toward post-quantum cryptography is:

An attacker can capture encrypted traffic today and potentially decrypt it later if a sufficiently powerful quantum computer becomes available.

For example:



For information that must remain confidential for decades, this is particularly important.

Examples include:

- Government records
- Financial information
- Healthcare records
- Intellectual property
- Long-lived credentials
- Industrial secrets
- Sensitive communications

ML-KEM helps address the **key-establishment** portion of this problem.

4. ML-KEM in Java 24

Java 24 provides standard APIs instead of requiring an external cryptographic library for ML-KEM.

For example:

```
KeyPairGenerator kpg =  
    KeyPairGenerator.getInstance("ML-KEM");  
  
kpg.initialize(NamedParameterSpec.ML_KEM_768);  
  
KeyPair keyPair = kpg.generateKeyPair();
```

The JDK supports:

ML-KEM-512
ML-KEM-768
ML-KEM-1024

These parameter sets are exposed through NamedParameterSpec. ([Oracle Docs](#))

5. ML-KEM Parameter Sets

Parameter Set	Relative Security	Relative Performance	Typical Consideration
ML-KEM-512	Lowest of the three	Generally fastest/smallest	Lower security requirements
ML-KEM-768	Medium	Balanced	Good general-purpose choice
ML-KEM-1024	Highest	Generally more computational/data	High-security applications

		overhead	
--	--	----------	--

The JDK supports all three parameter sets. ([Oracle Docs](#))

Important: Don't select a parameter set purely based on "higher is always better." Security requirements, interoperability, performance, message sizes, and organizational standards should determine the choice.

6. Advantages

Advantage	Explanation
Post-quantum security	Designed to resist attacks from sufficiently capable quantum computers
Standardized algorithm	Based on NIST FIPS 203
Built into Java 24	No external PQC library is required for the JDK's implementation
Standard Java APIs	Uses familiar JCA/JCE-style APIs
Three security levels	512, 768, and 1024 parameter sets
Good for future-proofing	Helps applications prepare for post-quantum migration
Useful for key establishment	Establishes shared secrets over insecure networks
Works with symmetric encryption	Can establish a key used by AES-GCM, etc.
Enterprise-friendly	Available through standard JDK cryptographic infrastructure
Reduces third-party dependency	Basic ML-KEM capability doesn't require a separate crypto provider
Supports crypto-agility planning	Applications can begin designing for PQC migration
Important for long-lived secrets	Helps address "harvest now, decrypt later" concerns

7. Disadvantages

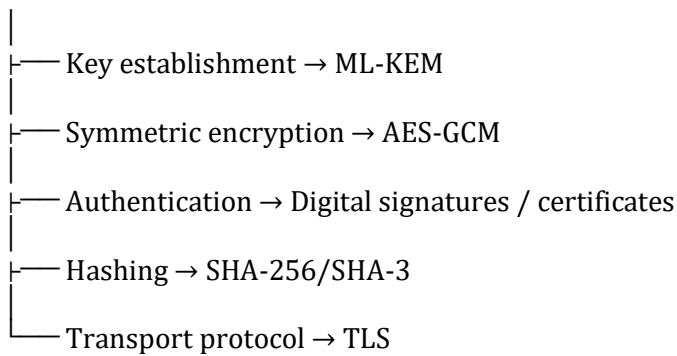
Disadvantage	Explanation
Not a complete encryption solution	ML-KEM establishes keys; it isn't a general replacement for AES-GCM
Larger keys/ciphertexts than many classical schemes	Post-quantum algorithms generally have larger data structures
Additional computational overhead	Performance characteristics differ from classical ECDH/X25519
Migration complexity	Existing protocols and systems need changes
Interoperability requirements	Both endpoints need compatible ML-KEM support/protocols
Does not replace digital signatures	Use ML-DSA or another appropriate signature algorithm for signatures
Protocol integration is still required	Merely generating an ML-KEM key pair doesn't make an application quantum-safe
Hybrid deployment may be necessary	Organizations may need classical + PQC mechanisms during migration
Not every external system supports ML-KEM	Legacy systems may only support RSA/ECDH/etc.
Parameter selection matters	Wrong security/performance selection can cause problems
Potential ecosystem limitations	TLS, certificates, HSMs, gateways, and other infrastructure need compatible support
Java version dependency	Standard JDK support starts with Java 24

8. Most Important Limitation

ML-KEM alone does **not** make an application completely post-quantum secure.

For example:

Application



ML-KEM addresses primarily:

Key establishment

while ML-DSA addresses:

Digital signatures

Java 24 introduced both ML-KEM and ML-DSA as separate JEPs. ([Oracle Docs](#))

9. When to Use

Situation	Recommendation	Reason
Long-lived confidential data	🔒 Excellent	Protect against future quantum threats
Government systems	🔒 Strong candidate	Long-term confidentiality requirements
Financial infrastructure	🔒 Strong candidate	Sensitive long-lived data
Healthcare systems	🔒 Strong candidate	Long-lived sensitive information
Secure messaging	🔒 Good	Quantum-resistant key establishment
VPN/network security	🔒 Good	Key establishment is central
Data-center communication	🔒 Good	Long-term confidentiality
High-value intellectual property	🔒 Good	Protect long-lived secrets

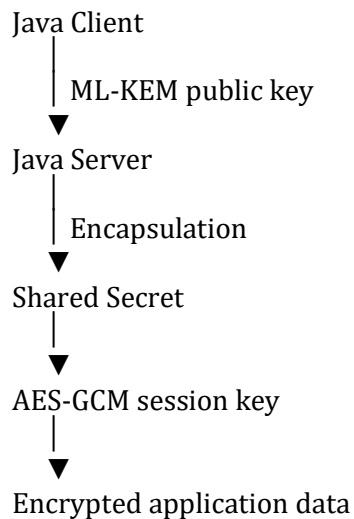
New cryptographic protocol	☑ Consider early	Avoid classical-only design
Migration from ECDH/DH	☑ Strong candidate	PQC transition
Short-lived non-sensitive data	☑ Lower priority	Risk/benefit may not justify migration yet
Legacy endpoint with no ML-KEM support	☑ Hybrid strategy	Compatibility required

10. When NOT to Use

Situation	Why?	Better Approach
You need digital signatures	ML-KEM isn't a signature algorithm	ML-DSA / appropriate signature algorithm
You need bulk data encryption	ML-KEM isn't designed for bulk encryption	AES-GCM/ChaCha20-Poly1305
Both endpoints don't support ML-KEM	Interoperability problem	Hybrid/classical mechanism
Data has very short confidentiality lifetime	PQC migration may have lower immediate value	Existing approved crypto, depending on policy
You only need hashing	ML-KEM is unnecessary	SHA-256/SHA-3
You need authentication only	KEM isn't authentication	Signature/certificate mechanism
Protocol doesn't support KEM	Cannot simply insert ML-KEM into it	Upgrade protocol or use supported hybrid
Regulatory requirements specify another mechanism	Compliance takes precedence	Approved organizational algorithm suite
You haven't designed key lifecycle/rotation	Algorithm alone isn't enough	Design complete key-management architecture

11. Real-Time Use Case — Secure Client/Server Key Establishment

Imagine:



Conceptually:

```
KeyPairGenerator kpg =  
    KeyPairGenerator.getInstance("ML-KEM");  
  
kpg.initialize(NamedParameterSpec.ML_KEM_768);  
  
KeyPair serverKeyPair =  
    kpg.generateKeyPair();
```

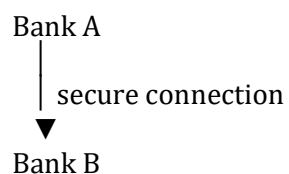
The server publishes its ML-KEM public key through the appropriate secure protocol.

The client can then use the KEM operation to establish a shared secret.

The JDK's ML-KEM implementation provides `KeyPairGenerator`, `KeyFactory`, and KEM support. (seanjmullan.org)

12. Real-Time Use Case — Financial Systems

Consider:



The information might need to remain confidential for many years.

Potential architecture:

ML-KEM
↓
Shared session secret
↓
AES-GCM
↓
Financial transaction data

Why use ML-KEM?

Long-lived sensitive data
+
Future quantum threat
↓
PQC migration becomes important

For actual financial deployments, algorithm selection should follow the organization's cryptographic standards and regulatory requirements.

13. Real-Time Use Case — Healthcare

Medical information can have a long confidentiality lifetime.

For example:

Hospital
|
patient information
▼
Insurance Provider

A future-oriented architecture could use post-quantum key establishment for appropriate communication channels.

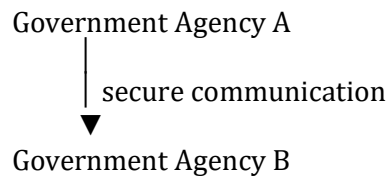
ML-KEM
↓
Shared key
↓
Authenticated encryption
↓
Medical data

The goal is to reduce the risk associated with storing encrypted traffic today that could potentially be targeted by future quantum capabilities.

14. Real-Time Use Case — Government Communication

Government data often has a long retention period.

Example:



Potential cryptographic architecture:

ML-KEM
+
ML-DSA
+
AES-GCM
+
Secure protocol

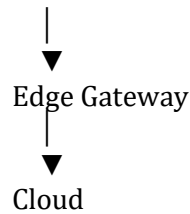
Here:

Requirement	Algorithm
Key establishment	ML-KEM
Digital signatures	ML-DSA
Bulk encryption	AES-GCM
Hashing	SHA-256/SHA-3, according to policy

15. Real-Time Use Case — IoT / Edge Devices

Suppose:

IoT Device



Devices may remain deployed for:

5–15+ years

A cryptographic algorithm chosen today could remain in service for a very long time.

Therefore:

Long device lifetime



PQC planning

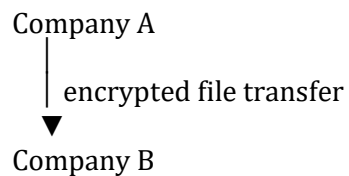


ML-KEM-capable architecture

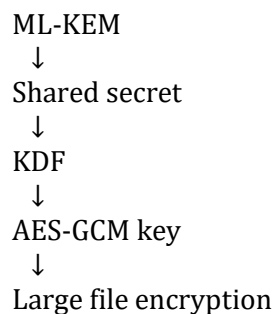
However, constrained devices need careful benchmarking because post-quantum cryptography can have larger key/ciphertext sizes and different computational characteristics.

16. Real-Time Use Case — Secure File Transfer

Imagine:



ML-KEM can be used to establish a session secret:



Do not use ML-KEM to encrypt the entire large file directly.

Instead:

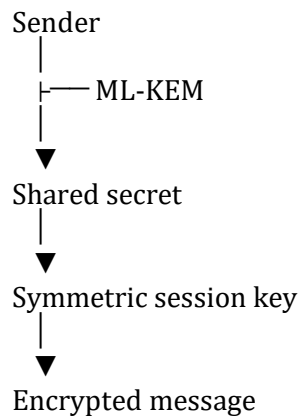
ML-KEM → establish key

AES-GCM → encrypt data

This hybrid design is much more appropriate.

17. Real-Time Use Case — Secure Messaging

Potential architecture:



For production messaging protocols, additional properties such as:

- authentication
- forward secrecy
- key rotation
- replay protection
- identity verification
- compromise recovery

must also be addressed.

ML-KEM alone doesn't provide the entire messaging-security architecture.

18. Java API Example

A simplified Java 24 example:

```
import java.security.KeyPair;
```

```

import java.security.KeyPairGenerator;
import java.security.spec.NamedParameterSpec;

public class MlKemExample {

    public static void main(String[] args)
        throws Exception {

        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("ML-KEM");

        generator.initialize(
            NamedParameterSpec.ML_KEM_768
        );

        KeyPair keyPair =
            generator.generateKeyPair();

        System.out.println(
            keyPair.getPublic().getAlgorithm()
        );
    }
}

```

The standard algorithm name is:

ML-KEM

and Java 24 supports:

ML-KEM-512
 ML-KEM-768
 ML-KEM-1024

according to the Java Security Standard Algorithm Names specification. ([Oracle Docs](#))

19. Conceptual KEM Flow

The KEM operation is different from normal symmetric encryption.

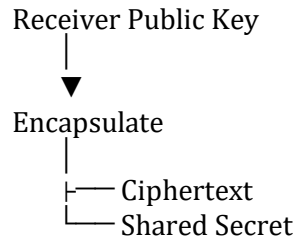
Receiver

```

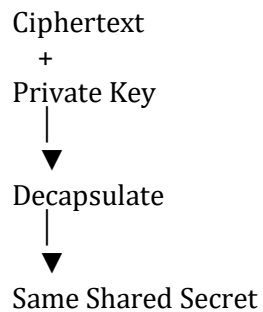
GenerateKeyPair()
|
├── Public Key
└── Private Key

```

Sender



Receiver



Therefore:

Sender shared secret
==
Receiver shared secret

That shared secret can then feed a symmetric encryption/KDF process.

20. ML-KEM vs RSA

Aspect	ML-KEM	RSA
Type	KEM	Public-key encryption/signature system
Quantum resistant	Designed for PQC	✗ No
Primary purpose	Key encapsulation	Encryption/signatures
Standard	NIST FIPS 203	Various standards
Security basis	Module lattices	Integer factorization
Future quantum resistance	☑ Designed for it	☑ Vulnerable to Shor's algorithm

Java 24 support	✓	✓
Best future-oriented use	Key establishment	Legacy/classical compatibility

21. ML-KEM vs ECDH/X25519

This is one of the most relevant comparisons.

Aspect	ML-KEM	X25519/ECDH
Purpose	Key establishment	Key agreement
Quantum resistant	☑ Designed for PQC	☑ No
Mathematical basis	Lattice	Elliptic curve
Classical security	N/A	☑ Strong against classical threats
Quantum resistance	☑	☑
Key/ciphertext size	Generally larger	Generally smaller
Ecosystem maturity	Newer	Very mature
Legacy compatibility	Lower	Very high
PQC migration	☑	☑ Not sufficient alone

22. ML-KEM vs ML-DSA

These two Java 24 features are complementary.

Aspect	ML-KEM	ML-DSA
JEP	496	497
Purpose	Key establishment	Digital signatures
NIST standard	FIPS 203	FIPS 204

Generates key pair	✓	✓
Establishes shared secret	✓	✗
Signs data	✗	✓
Verifies signature	✗	✓
Quantum resistant	?	?
Primary use	Secure communication keys	Authentication/integrity
Java 24	Final	Final

Oracle explicitly describes ML-KEM as protecting symmetric keys and ML-DSA as providing digital signatures for detecting unauthorized modification and authenticating signatories. ([Oracle Docs](#))

23. Alternate Features

There isn't a single exact replacement for ML-KEM. The alternatives depend on the requirement.

Alternative	Purpose	Quantum Resistant?	When to Use
X25519	Key agreement	✗	Classical cryptography
ECDH	Key agreement	✗	Existing classical systems
Diffie-Hellman	Key agreement	✗	Legacy/classical systems
RSA	Encryption/key transport	✗	Legacy interoperability
ML-KEM	PQ key encapsulation	✓	Post-quantum migration
Hybrid classical + ML-KEM	Combined security	?	Migration/interoperability
HPKE	Hybrid public-key encryption framework	Depends on algorithms	Protocol designs requiring HPKE

Pre-shared keys	Symmetric key establishment	Depends on implementation	Controlled environments

24. Best Alternative — Hybrid Cryptography

During migration, many organizations may prefer:

Classical KEM
 +
 ML-KEM
 ↓
 Combined shared secret

For example:

X25519
 +
 ML-KEM-768
 ↓
 KDF
 ↓
 Session key

Why?

Classical security
 +
 Post-quantum security
 ↓
 Migration resilience

This can be valuable while the PQC ecosystem is still being integrated across:

- clients
- servers
- TLS
- proxies
- HSMs
- certificates
- gateways
- cloud services
- network appliances

The exact hybrid construction must follow a standardized protocol/design; **do not invent your own way of combining secrets**.

Oracle has also described ongoing work toward post-quantum TLS and future Java PQC capabilities, illustrating that ML-KEM is one building block in a broader migration. ([Oracle Blogs](#))

25. ML-KEM vs Hybrid Approach

Aspect	ML-KEM Only	Classical + ML-KEM Hybrid
Quantum resistance	?	?
Classical algorithm fallback	✗	?
Legacy interoperability	?	?
Migration complexity	Medium	Higher
Protocol support required	High	Very high
Future migration value	?	? ?
Best for	PQC-native systems	Transition period

26. Performance Considerations

Area	Expected Impact
Key generation	More computational/data overhead than some classical alternatives
Encapsulation	Workload-dependent
Decapsulation	Workload-dependent
Key size	Generally larger than X25519-style classical keys
Ciphertext size	Generally larger
Bulk encryption	ML-KEM isn't used for bulk encryption

AES-GCM performance	Unchanged once the session key is established
Network bandwidth	Can be affected during KEM exchange
Memory	Potentially higher during KEM operations
JVM performance	Depends on provider/implementation/hardware

Do not assume ML-KEM is "slower" or "faster" universally. Benchmark your actual protocol and workload.

The most obvious practical cost is often **larger cryptographic data structures and handshake overhead**, rather than bulk data-encryption cost.

27. Security Architecture

A good architecture separates responsibilities:

Security Requirement	Recommended Primitive
Quantum-resistant key establishment	ML-KEM
Bulk encryption	AES-GCM / ChaCha20-Poly1305
Quantum-resistant signatures	ML-DSA
Key derivation	KDF
Hashing	SHA-256/SHA-3 according to requirements
Authentication	Certificates/signatures/protocol mechanisms
Transport security	TLS or another approved secure protocol

This is why adopting ML-KEM should be treated as part of a **cryptographic architecture migration**, not as simply replacing one Java API call.

28. Migration Strategy

Step	Action
------	--------

1	Inventory current RSA/DH/ECDH usage
2	Identify data requiring long-term confidentiality
3	Identify external systems and protocols
4	Determine PQC requirements
5	Evaluate ML-KEM parameter sets
6	Evaluate hybrid mechanisms
7	Update protocol/TLS infrastructure where supported
8	Benchmark handshake and network overhead
9	Test interoperability
10	Implement crypto-agility
11	Rotate/deprecate vulnerable classical mechanisms according to policy
12	Monitor evolving standards and vendor support

29. What Not to Do

Bad Practice	Problem
Use ML-KEM to encrypt huge files directly	Wrong abstraction
Replace every RSA key with ML-KEM blindly	RSA also serves signature use cases
Use ML-KEM as a digital signature	It isn't a signature algorithm
Invent your own hybrid construction	Can introduce serious cryptographic vulnerabilities
Ignore key rotation	Algorithm alone doesn't provide lifecycle security
Ignore authentication	Key establishment alone doesn't authenticate identities
Assume ML-KEM makes TLS automatically	Protocol support is also required

quantum-safe	
Select ML-KEM-1024 simply because it is "largest"	Security requirements and performance matter
Hard-code a provider unnecessarily	Can reduce crypto agility
Skip interoperability testing	PQC ecosystem support varies

30. Best Practices

Practice	Recommendation
Prefer standard JDK APIs	❓
Use NIST-standardized ML-KEM	❓
Use ML-KEM for key establishment	❓
Use AES-GCM/another approved AEAD for bulk data	❓
Use ML-DSA for PQ signatures where appropriate	❓
Design for crypto agility	❓
Benchmark parameter sets	❓
Consider standardized hybrid mechanisms	❓
Protect private keys	❓ Critical
Authenticate endpoints	❓ Critical
Don't invent cryptographic protocols	❓ Critical
Follow organizational/regulatory standards	❓ Critical

31. Decision Table

Question	Recommendation
Does the application require long-term confidentiality?	☑ Consider ML-KEM
Are you designing a new secure protocol?	☑ Consider PQC from the beginning
Are you replacing ECDH/DH for PQ readiness?	☑ ML-KEM is a strong candidate
Do you need digital signatures?	☑ ML-DSA, not ML-KEM
Do you need bulk encryption?	☑ AES-GCM/ChaCha20-Poly1305
Does the other endpoint support only X25519/ECDH?	☑ Consider standardized hybrid/transition strategy
Is the data low-value and short-lived?	☑ Lower migration priority
Are you building a healthcare/financial/government system?	☑ Strongly evaluate PQC
Do you have strict cryptographic regulations?	☑ Follow approved algorithm suites
Are you simply trying to make Java code faster?	☑ ML-KEM isn't a performance feature

32. Overall Advantages vs Disadvantages

Category	Assessment
Future quantum resistance	☑ Excellent
Standardization	☑ Excellent
Java integration	☑ Excellent
Long-term confidentiality	☑ Excellent
Key establishment	☑ Excellent
Digital signatures	☑ Not applicable

Bulk encryption	❑ Not applicable
Classical interoperability	❑
Network overhead	❑ Higher than many classical schemes
Migration complexity	❑ Medium/High
Enterprise relevance	❑ Very high
Immediate benefit for low-value data	❑ Low
Future-proofing	❑ Excellent

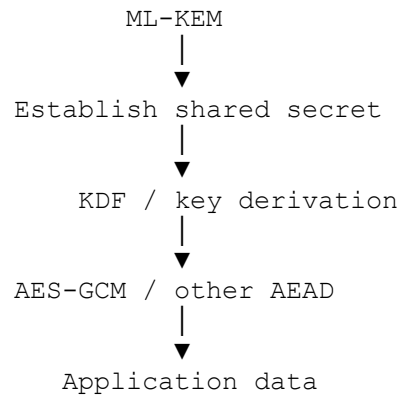
33. Final Recommendation

Scenario	Best Choice
New secure communication system	★ Evaluate ML-KEM
Long-term confidential data	★ Strong ML-KEM candidate
Financial/government/healthcare	★ Strongly evaluate PQC
Need quantum-resistant signatures	★ ML-DSA
Bulk data encryption	★ AES-GCM/approved AEAD
Existing X25519 infrastructure	★ Consider standardized hybrid migration
Legacy endpoint without PQC	★ Hybrid/transition approach
Simple non-sensitive application	❑ Lower priority
Only need authentication/signatures	❑ ML-DSA/signature mechanism
Need a complete PQ security solution	★ ML-KEM + appropriate signatures + AEAD + secure protocol

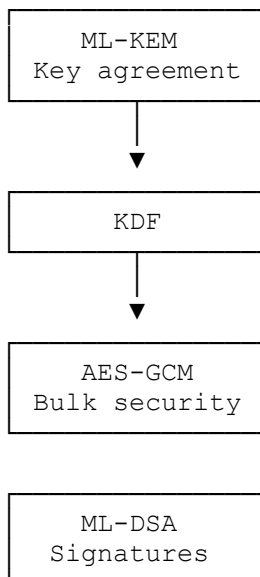
Bottom line

Java 24's ML-KEM support is one of the most strategically important security features in the release. Unlike many Java 24 features that improve syntax or performance, JEP 496 addresses a fundamental future-security problem.

The correct mental model is:



And for a complete post-quantum security architecture:



So, use ML-KEM when you need quantum-resistant key establishment, especially for systems whose confidential data has a long lifetime. Do not treat ML-KEM as a replacement for encryption, signatures, TLS, authentication, or key management.

Java 24 provides **ML-KEM-512**, **ML-KEM-768**, and **ML-KEM-1024** through standard cryptographic APIs, making PQC experimentation and migration significantly easier without requiring an external provider for the basic implementation. ([Oracle Docs](#))

Java 24 — Quantum-Resistant ML-DSA

Property	Details
Java Version	24
Year	2025
Feature	Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm (ML-DSA)
JEP	JEP 497
UniqueClassName	QuantumResistantMLDSA
Status	Final
Standard	NIST FIPS 204
Security category	Post-Quantum Cryptography (PQC)
Primary purpose	Quantum-resistant digital signatures
Java APIs	KeyPairGenerator, Signature, KeyFactory
Provider	SUN
Parameter sets	ML-DSA-44, ML-DSA-65, ML-DSA-87
Main security properties	Authentication, integrity, non-repudiation-style signature use cases
Preview?	✗ No — final feature

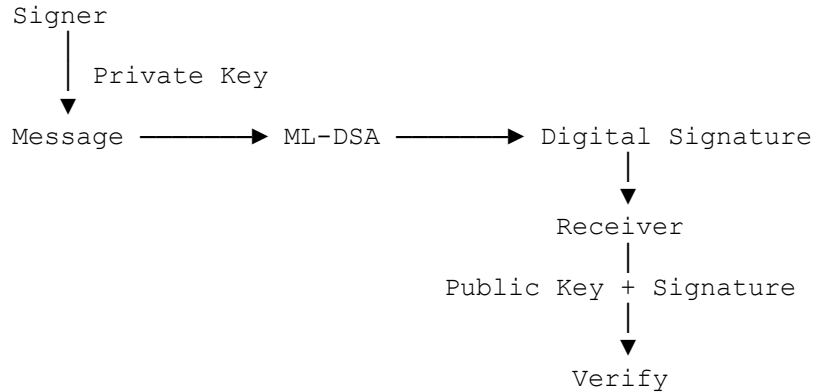
Java 24 added ML-DSA as a standard JDK implementation. It is based on lattices and standardized by NIST in **FIPS 204**. Oracle describes its purpose as detecting unauthorized modification of data and authenticating the identity of signatories while providing resistance against future quantum-computing attacks. ([Oracle](#))

1. What Is ML-DSA?

ML-DSA = Module-Lattice-Based Digital Signature Algorithm.

It is a **digital signature algorithm**, not an encryption algorithm and not a key-exchange algorithm.

Its basic purpose is:



The receiver can determine:

1. Whether the message was modified.
2. Whether the signature corresponds to the expected public key.
3. Whether the signer possessed the corresponding private key.

ML-DSA is standardized as **FIPS 204** and Java 24 supports it through the standard Java security APIs. ([Oracle Docs](#))

2. Why Is ML-DSA "Quantum-Resistant"?

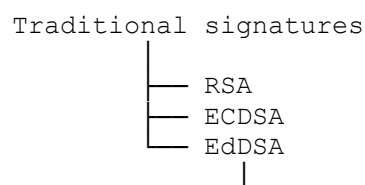
Traditional digital-signature algorithms include:

RSA
ECDSA
Ed25519
DSA

Some widely deployed public-key signature systems are based on mathematical problems that sufficiently capable quantum computers are expected to threaten.

ML-DSA uses a **lattice-based construction** designed to withstand attacks from quantum computers.

Conceptually:



▼
Quantum vulnerable

versus:

Post-quantum signatures
└─ ML-DSA
 ▼
 Lattice-based
 cryptography

Oracle identifies ML-DSA as one of Java 24's major steps toward post-quantum cryptographic readiness. ([Oracle](#))

3. ML-DSA Does NOT Encrypt Data

This distinction is critical.

Requirement	ML-DSA?	Appropriate technology
Digital signature	✓	ML-DSA
Verify authenticity	✓	ML-DSA
Detect modification	✓	ML-DSA
Establish shared secret	✗	ML-KEM
Encrypt bulk data	✗	AES-GCM / ChaCha20-Poly1305
Hash data	✗	SHA-256/SHA-3
Key derivation	✗	KDF/HKDF
Quantum-resistant key establishment	✗	ML-KEM

The simplest mental model is:

ML-KEM → "How do we establish a secret?"

ML-DSA → "How do we prove who signed this and that it wasn't changed?"

4. Basic Signing Flow

Signer

Message
+
ML-DSA Private Key
↓
▼
Digital Signature

Receiver

Message
+
Digital Signature
+
ML-DSA Public Key
↓
▼
Verification
├─ true → authentic/integrity verified
└─ false → reject

5. Simple Java 24 Example

Java 24 exposes ML-DSA through familiar JCA APIs.

```
import java.nio.charset.StandardCharsets;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.Signature;
import java.security.spec.NamedParameterSpec;

public class MLDsaExample {

    public static void main(String[] args)
        throws Exception {

        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("ML-DSA");

        generator.initialize(
            NamedParameterSpec.ML_DSA_65
        );

        KeyPair keyPair =
```

```

        generator.generateKeyPair();

byte[] message =
    "Important transaction".getBytes(
        StandardCharsets.UTF_8
    );

Signature signer =
    Signature.getInstance("ML-DSA");

signer.initSign(keyPair.getPrivate());
signer.update(message);

byte[] signature =
    signer.sign();

Signature verifier =
    Signature.getInstance("ML-DSA");

verifier.initVerify(keyPair.getPublic());
verifier.update(message);

boolean valid =
    verifier.verify(signature);

System.out.println(
    "Signature valid = " + valid
);
}
}

```

Java 24 provides KeyPairGenerator, KeyFactory, and Signature implementations for ML-DSA. (seanjmullan.org)

6. ML-DSA Parameter Sets

Java 24 supports three parameter sets:

Parameter Set	Security Level	Relative Performance	Typical Use
ML-DSA-44	Lowest of the three	Generally fastest/smallest	Applications with lower security requirements
ML-DSA-65	Medium	Balanced	General-purpose choice

ML-DSA-87	Highest	Generally more overhead	High-security applications

The Java 24 API exposes these through `NamedParameterSpec.ML_DSA_44`, `ML_DSA_65`, and `ML_DSA_87`. ([Oracle Docs](#))

Do not automatically select ML-DSA-87 simply because it is the largest. The appropriate parameter set should follow your security policy, interoperability requirements, and performance constraints.

7. Advantages

Advantage	Explanation
Quantum-resistant design	Designed to resist future quantum-computing attacks
NIST standardized	Based on FIPS 204
Built into Java 24	No external PQC provider is required for the standard implementation
Standard Java APIs	Uses familiar <code>Signature</code> , <code>KeyPairGenerator</code> , and <code>KeyFactory</code> APIs
Three parameter sets	44, 65 and 87 provide different security/performance levels
Authentication	Can establish cryptographic evidence associated with a signing key
Integrity protection	Detects modification of signed data
Future-proofing	Helps prepare systems for post-quantum cryptography
Suitable for long-lived signatures	Useful where signatures may need verification many years into the future
Reduces third-party dependency	Basic ML-DSA functionality is available in the JDK
Fits existing JCA architecture	Easier migration from other signature algorithms

Useful for software signing	Can protect software/artifacts against tampering
Useful for document signing	Can provide quantum-resistant signatures
Useful for certificates/protocols	Potential building block for future PQ authentication infrastructure

Oracle specifically identifies ML-DSA as a Java 24 security enhancement for detecting unauthorized modification and authenticating signatories. ([Oracle](#))

8. Disadvantages

Disadvantage	Explanation
Larger signatures	ML-DSA signatures are generally much larger than compact classical signatures such as Ed25519
Larger public/private keys	Key material is larger than many classical alternatives
Higher bandwidth requirements	Large signatures matter in network protocols and signed messages
Higher storage requirements	Important when storing millions of signatures
Migration complexity	Existing certificates, protocols, HSMs and infrastructure may need changes
Interoperability limitations	All parties must support the algorithm and encoding
Does not provide encryption	It only provides digital signatures
Does not establish shared secrets	Use ML-KEM for that purpose
Not automatically supported by every protocol	Protocol-level integration is required
Certificate ecosystem changes	PKI and certificate tooling need ML-DSA support
Potential performance overhead	Key generation/signing/verification characteristics differ from classical algorithms

Long-term compatibility concerns	PQC standards and ecosystem integration continue to evolve
Private-key protection remains essential	Quantum resistance doesn't help if the private key is stolen
Signature migration is non-trivial	Existing signed artifacts may require dual-signature or migration strategies

9. When to Use

Situation	Recommendation	Reason
Long-lived digital signatures	👍 Excellent	Future quantum resistance
Government documents	👍 Strong candidate	Long retention periods
Financial documents	👍 Strong candidate	Long-term authenticity
Software/code signing	👍 Strong candidate	Protect software integrity
Firmware signing	👍 Strong candidate	Devices may remain deployed for years
Secure document signing	👍 Excellent	Long-term signature verification
Supply-chain security	👍 Excellent	Verify software/artifact provenance
New security-sensitive systems	👍 Consider early	Avoid classical-only architecture
Public-key authentication	👍 Good	Quantum-resistant signatures
Certificate infrastructure	👍 Evaluate	Depends on ecosystem support
Short-lived low-value data	👍 Lower priority	Migration cost may exceed immediate benefit
Legacy interoperability	👍 Hybrid may be better	Existing endpoints may not support ML-DSA

10. When NOT to Use

Situation	Why Avoid It?	Better Choice
Need encryption	ML-DSA is not encryption	AES-GCM / ChaCha20-Poly1305
Need key establishment	ML-DSA doesn't establish shared secrets	ML-KEM
Need hashing	Signature algorithm is unnecessary	SHA-256/SHA-3
Need very small signatures	ML-DSA signatures are relatively large	Classical algorithm if PQC isn't required
Endpoint doesn't support ML-DSA	Interoperability failure	Compatible signature / hybrid strategy
Existing protocol doesn't support it	Cannot simply substitute it	Protocol upgrade
Low-value short-lived data	PQC migration may not be worthwhile	Existing approved algorithm
Regulatory policy requires another algorithm	Compliance takes precedence	Approved algorithm
You only need data confidentiality	Signature isn't enough	AEAD encryption
You haven't designed key management	Algorithm alone doesn't provide security	Complete PKI/key-management design

11. Real-Time Use Case — Software Signing

A software vendor publishes:

application.jar

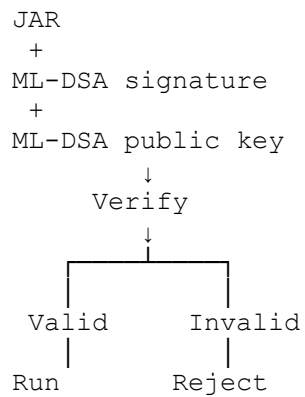
An attacker modifies it:

application.jar
↓
Malicious modification

A user downloads:

application.jar
+
ML-DSA signature

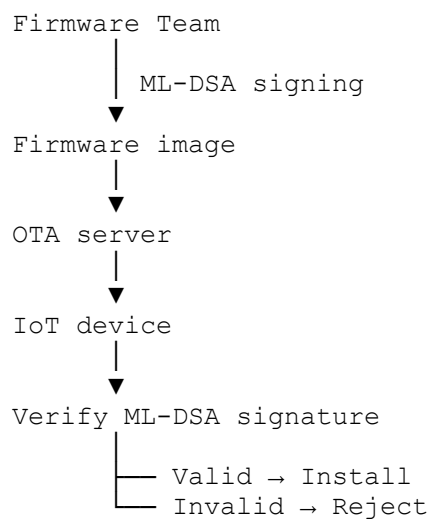
Verification:



This is particularly valuable for software expected to remain trusted for many years.

12. Real-Time Use Case — Firmware Signing

Consider an IoT device:



This is a strong use case because:

- Devices may remain deployed for many years.
 - Firmware must be authenticated.
 - A compromised signing mechanism can affect a large device population.
 - Post-quantum migration is easier when designed into the device lifecycle early.
-

13. Real-Time Use Case — Financial Document

Suppose a bank issues a digitally signed document:

LoanAgreement.pdf

+

ML-DSA signature

Years later:

Document

+

Signature

+

Public Key

↓

Verification

The verifier can determine whether the signed content has changed and whether the signature corresponds to the signing key.

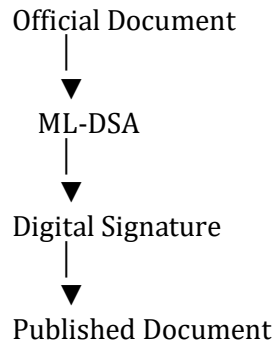
This is especially useful for documents that must remain verifiable for long periods.

14. Real-Time Use Case — Government Records

A government agency may digitally sign:

- certificates
- licenses
- legal documents
- approvals
- records
- official communications

Architecture:



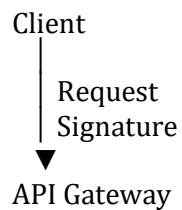
Years later:



This is exactly the type of long-lived authenticity use case where post-quantum signatures can be valuable.

15. Real-Time Use Case — API Request Signing

Consider:



The client signs:

HTTP method
+
URI
+
timestamp
+
body hash

with its ML-DSA private key.

The server:

Request

```
+  
Signature  
+  
Client Public Key  
↓  
Verify  
↓  
Accept / Reject
```

A real implementation should define canonicalization, replay protection, timestamps/nonces, key rotation, and failure handling. **Do not invent a cryptographic protocol casually.**

16. Real-Time Use Case — Software Supply Chain

Consider:

```
Developer  
↓  
Build  
↓  
Artifact  
↓  
ML-DSA Sign  
↓  
Artifact Repository  
↓  
Deployment Platform  
↓  
ML-DSA Verify
```

For example:

```
payment-service.jar  
payment-service.jar.sig
```

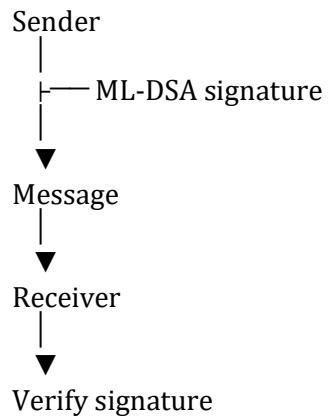
The deployment platform verifies:

```
Signature  
↓  
Correct signing key?  
↓  
Artifact unchanged?  
↓  
Trusted publisher?
```

This can be combined with SBOMs, provenance metadata, trusted build systems, and artifact repositories.

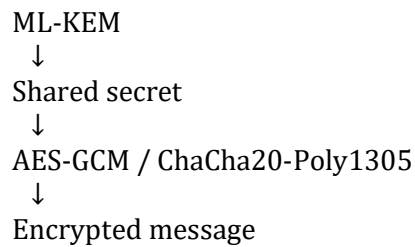
17. Real-Time Use Case — Secure Messaging

For a messaging system:



ML-DSA provides the signature component.

For confidentiality, you would separately use:



Therefore a post-quantum messaging architecture could conceptually be:

ML-KEM → Key establishment
ML-DSA → Authentication/signature
AEAD → Message encryption

18. ML-DSA vs ML-KEM

These two Java 24 features should be understood together.

Aspect	ML-DSA	ML-KEM
JEP	497	496

NIST standard	FIPS 204	FIPS 203
Primary purpose	Digital signatures	Key encapsulation
Quantum-resistant	✓	✓
Signs data	✓	✗
Verifies signatures	✓	✗
Establishes shared secret	✗	✓
Authentication	✓	Not by itself
Integrity	✓	Not its primary purpose
Bulk encryption	✗	✗
Typical combination	ML-KEM + ML-DSA	ML-DSA + ML-KEM

The simplest rule:

ML-KEM → "Establish the secret."

ML-DSA → "Prove who signed it / detect modification."

Oracle explicitly distinguishes the two in Java 24's security enhancements. ([Oracle](#))

19. ML-DSA vs RSA

Aspect	ML-DSA	RSA
Digital signatures	✓	✓
Quantum-resistant	☑ Designed for PQC	☑ No
Mathematical basis	Lattice	Integer factorization
NIST standard	FIPS 204	RSA standards
Signature size	Larger	Generally smaller
Key size	Larger	Generally smaller
Ecosystem maturity	Newer	Very mature

Legacy interoperability	Lower	Excellent
Future quantum security	?	?
Best role	PQ migration	Existing/classical systems

20. ML-DSA vs ECDSA

Aspect	ML-DSA	ECDSA
Digital signatures	✓	✓
Quantum resistant	?	?
Mathematical basis	Lattice	Elliptic curve
Signature size	Generally larger	Smaller
Key size	Generally larger	Smaller
Existing ecosystem	Growing	Very mature
Mobile/embedded bandwidth	? More challenging	? Excellent
Long-term PQ readiness	?	?
Migration cost	Higher	Low
Best role	Post-quantum transition	Existing classical systems

21. ML-DSA vs Ed25519

Aspect	ML-DSA	Ed25519
Type	PQ digital signature	Classical digital signature
Quantum resistance	✓	✗
Key size	Larger	Very small

Signature size	Larger	64 bytes
Performance	Workload-dependent	Generally very fast
Network overhead	Higher	Very low
Ecosystem maturity	Newer	Very mature
Long-term PQ security	?	?
Best use	PQ migration	Classical applications

22. Alternate Features

There is no single exact replacement for ML-DSA because the appropriate alternative depends on the required security property.

Alternative	Type	Quantum Resistant?	Best Use
Ed25519	Digital signature	✗	Modern classical systems
Ed448	Digital signature	✗	Higher classical security
ECDSA	Digital signature	✗	Existing PKI/TLS ecosystem
RSA-PSS	Digital signature	✗	Enterprise/legacy compatibility
RSA	Digital signature	✗	Legacy systems
ML-DSA	Digital signature	✓	Post-quantum signatures
SLH-DSA	Hash-based signature	✓	Alternative PQ signature approach
LMS/HSS	Hash-based signature	✓	Specialized stateful signature systems
Hybrid classical + PQ signature	Combined	?	Migration/interoperability

23. ML-DSA vs SLH-DSA

SLH-DSA is particularly interesting as an alternative post-quantum signature algorithm.

Aspect	ML-DSA	SLH-DSA
Type	Lattice-based	Hash-based
Quantum resistant	✓	✓
NIST standard	FIPS 204	FIPS 205
Security assumption	Lattice problems	Hash functions
Signature size	Generally smaller than SLH-DSA	Generally much larger
Performance	Generally attractive	Different performance trade-offs
Key sizes	Moderate	Small/moderate
Signature sizes	Moderate	Large
Stateful?	No	No
Best use	General-purpose PQ signatures	Alternative PQ assurance/diversity

Why might an organization care about both?

Cryptographic diversity can reduce dependence on a single mathematical assumption.

24. ML-DSA vs Hash-Based Signatures

Requirement	ML-DSA	Hash-Based Signature
General-purpose PQ signature	★★★★★	★★★
Signature size	Better	Often worse
Performance	Good	Workload-dependent
Conservative security assumption	★★★★★	★★★★★

Stateful concerns	No	Some schemes are stateful; SLH-DSA is stateless
Ecosystem	Growing	Growing
Best general choice	?	?

25. Hybrid Signatures

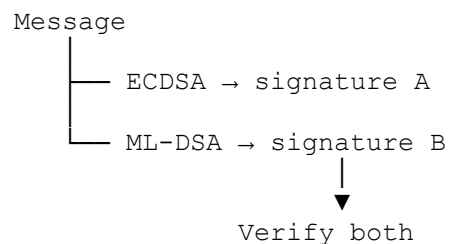
During migration, organizations may use:

Classical Signature
 +
 Post-Quantum Signature
 ↓
 Combined verification

For example:

ECDSA
 +
 ML-DSA
 ↓
 Hybrid authentication

Conceptually:



This can provide a transition path while legacy systems still require classical signatures.

However, **the exact hybrid construction should follow an established protocol/standard rather than being designed ad hoc.**

26. ML-DSA vs Hybrid Approach

Aspect	ML-DSA Only	Classical + ML-DSA
PQ security	?	?
Legacy compatibility	?	?
Signature overhead	Lower	Higher
Migration complexity	Medium	High
Existing infrastructure	May require changes	Easier transition
Long-term PQ readiness	?	?
Best for new PQ-native system	★★★★★	★★★★
Best for transition	★★★	★★★★★

27. Performance Considerations

*ML-DSA should **not** be evaluated solely by raw CPU time.*

Area	Expected Consideration
Key generation	More involved than many classical schemes
Signing	Workload-dependent
Verification	Workload-dependent
Public key size	Larger than many classical signatures
Private key size	Larger
Signature size	Larger
Network bandwidth	Potentially significant
Storage	Higher
JVM execution	Depends on implementation/hardware

Bulk encryption	No effect — ML-DSA isn't used for bulk encryption
------------------------	---

The biggest architectural difference often comes from **larger keys and signatures**, especially when signing many small messages or transmitting signatures over bandwidth-constrained links.

Therefore:

Benchmark your actual protocol, not just the `Signature.sign()` call.

28. Key Sizes and Signature Sizes Matter

For example, if an application signs millions of objects:

```

1 signature
  ↓
larger ML-DSA signature
  ↓
1 million signatures
  ↓
substantial additional storage

```

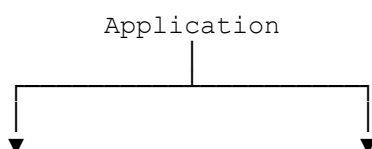
This matters for:

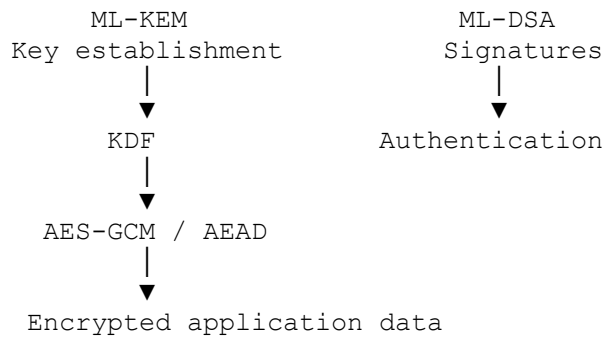
- Certificate infrastructures
- IoT
- Blockchain-like systems
- Artifact repositories
- Signed messages
- High-volume event streams
- Long-term archives

Therefore, PQC migration should consider **storage and network costs**, not just cryptographic security.

29. Security Architecture

A complete secure system could look like:





Responsibilities:

Security requirement	Technology
Quantum-resistant key establishment	ML-KEM
Quantum-resistant signatures	ML-DSA
Bulk encryption	AES-GCM / approved AEAD
Key derivation	KDF/HKDF
Hashing	SHA-256/SHA-3, as required
Identity	Certificates / signatures / protocol
Transport	TLS or approved secure protocol

30. Migration Strategy

Step	Action
1	Inventory RSA/ECDSA/EdDSA signatures
2	Identify long-lived signed data
3	Identify software/firmware signing
4	Identify certificates and PKI dependencies
5	Check HSM support
6	Check TLS/protocol support

7	Select ML-DSA parameter set
8	Benchmark key/signature sizes
9	Evaluate hybrid signatures
10	Build crypto-agility into applications
11	Test interoperability
12	Establish key rotation and revocation
13	Plan migration of existing signatures
14	Monitor standards and infrastructure support

31. What ML-DSA Does NOT Solve

Problem	ML-DSA Solution?
Data encryption	✗
Key exchange	✗
Confidentiality	✗
Password hashing	✗
Secure random generation	✗
Certificate lifecycle	✗
Key storage	✗
Key rotation	✗
Revocation	✗
Authentication protocol	☒ Only the signature primitive
Message integrity	✓
Signature verification	✓

Signer authentication	✓ when public-key trust is established
Post-quantum signature security	✓

This distinction prevents a common mistake:

ML-DSA is a cryptographic primitive, not a complete security architecture.

32. Best Practices

Practice	Recommendation
Use standard JDK APIs	?
Use NIST-standardized ML-DSA	?
Protect ML-DSA private keys	? Critical
Use appropriate parameter set	?
Consider long-lived data first	?
Build crypto agility	?
Benchmark signature size	?
Test interoperability	?
Consider hybrid migration	?
Use ML-KEM for key establishment	?
Use AEAD for bulk encryption	?
Don't invent hybrid protocols	? Critical
Don't treat ML-DSA as encryption	? Critical
Follow organizational crypto policy	? Critical

33. Decision Table

Question	Recommendation
Do you need quantum-resistant digital signatures?	🔍 ML-DSA
Do you sign software or firmware?	🔍 Strong candidate
Do signatures need to remain verifiable for decades?	🔍 Strong candidate
Are you designing a new secure system?	🔍 Evaluate now
Do you need quantum-resistant key establishment?	🔍 ML-KEM instead
Do you need bulk encryption?	🔍 AES-GCM/AEAD
Do you need classical interoperability?	🔍 Consider standardized hybrid approach
Do you need tiny signatures?	🔍 Classical signature may be more efficient if PQC isn't required
Is data short-lived and low-value?	🔍 Lower migration priority
Does infrastructure not support ML-DSA?	🔍 Plan migration/hybrid strategy
Do regulations specify another algorithm?	🔍 Follow applicable policy

34. Overall Advantages vs Disadvantages

Category	Assessment
Quantum resistance	🔍 Excellent
NIST standardization	🔍 Excellent
Java integration	🔍 Excellent
Software signing	🔍 Excellent
Firmware signing	🔍 Excellent
Long-term documents	🔍 Excellent
Authentication	🔍 Excellent

Data confidentiality	ⓧ Not applicable
Key establishment	ⓧ Not applicable
Signature size	ⓧ Larger
Network overhead	ⓧ Higher
Storage overhead	ⓧ Higher
Legacy compatibility	ⓧ Limited
Migration effort	ⓧ Medium/High
Enterprise relevance	ⓧ Very high
Future-proofing	ⓧ Excellent

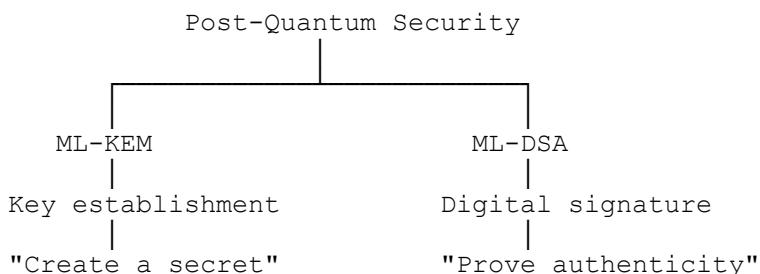
35. Final Recommendation

Scenario	Best Choice
Quantum-resistant signatures	★ ML-DSA
Software signing	★ ML-DSA
Firmware signing	★ ML-DSA
Long-lived documents	★ ML-DSA
Government/financial records	★ Strongly evaluate ML-DSA
Key establishment	★ ML-KEM
Bulk encryption	★ AES-GCM/approved AEAD
Classical + PQ transition	★ Standardized hybrid approach
Alternative PQ signature	★ SLH-DSA
Legacy classical system	★ RSA-PSS/ECDSA/Ed25519 as appropriate

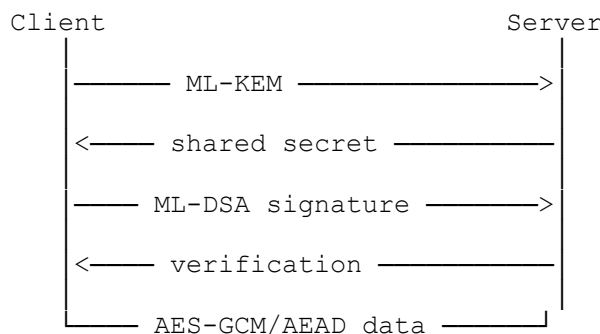
Bottom line

Java 24's ML-DSA is the digital-signature counterpart to ML-KEM.

Think of the two features this way:



A realistic secure architecture is therefore:



Use ML-DSA when your primary requirement is quantum-resistant authentication, integrity, and digital signatures—particularly for software/firmware signing, long-lived documents, government/financial records, and new security-sensitive systems.

Do not use ML-DSA as a replacement for ML-KEM or AES-GCM: ML-DSA signs; ML-KEM establishes secrets; symmetric AEAD encrypts the actual data.

Java 24 provides the three standardized parameter sets **ML-DSA-44, ML-DSA-65, and ML-DSA-87**, exposed through standard `KeyPairGenerator`, `Signature`, and `KeyFactory` APIs. ([Oracle Docs](#))

Java 24 — Unsafe Memory-Access Warnings

Property	Details
Java Version	24
Year	2025
Feature	Unsafe Memory-Access Warnings

JEP	JEP 498
UniqueClassName	UnsafeMemoryAccessWarnings
Status	Final
Affected API	sun.misc.Unsafe
Main change	Runtime warning when deprecated memory-access methods of sun.misc.Unsafe are invoked
Earlier change	These memory-access methods were deprecated for removal in Java 23
Main goal	Prepare applications and libraries to migrate away from unsupported Unsafe memory-access methods
Recommended replacements	VarHandle API and Foreign Function & Memory (FFM) API
Application-code change required?	Not necessarily immediately, but migration is strongly recommended

Java 24 does **not** remove sun.misc.Unsafe. Instead, JEP 498 makes use of its deprecated memory-access methods produce a runtime warning on the first invocation. Oracle recommends migrating to supported APIs such as VarHandle and the Foreign Function & Memory API. ([Oracle Docs](#))

1. What Problem Does It Solve?

sun.misc.Unsafe has historically allowed Java libraries to perform operations outside normal Java safety guarantees.

For example:

Unsafe unsafe;

has historically been used for:

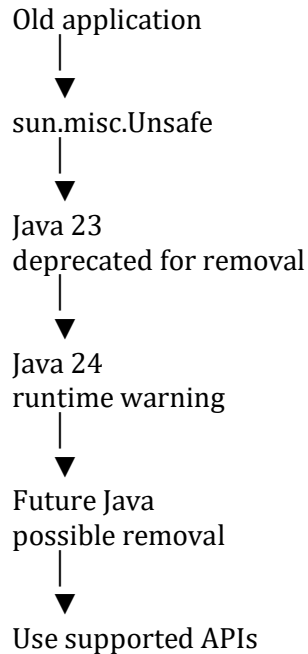
- Direct memory access
- CAS operations
- Memory ordering
- Object field manipulation
- Off-heap memory
- Low-level concurrency

- Performance-sensitive libraries

However, Unsafe is an **unsupported internal API**.

Java 24 starts making its eventual removal much more visible.

The migration path is:



Oracle explicitly states that the affected methods have been deprecated for removal and have been superseded by standard APIs. ([Oracle Docs](#))

2. What Exactly Causes the Warning?

The warning concerns **memory-access methods** in `sun.misc.Unsafe`, not every method in the class.

Conceptually:

```
unsafe.getInt(...);
unsafe.putInt(...);
unsafe.getLong(...);
unsafe.putLong(...);
unsafe.getObject(...);
unsafe.putObject(...);
```

and related memory-access operations are the focus.

Java 24 issues a warning the **first time a memory-access method is invoked**. ([Oracle Docs](#))

So:

```
unsafe.getInt(...);
```

may cause a warning such as conceptually:

WARNING: A restricted method in sun.misc.Unsafe has been called

The exact warning text depends on the method/context.

3. Why Is Unsafe Being Phased Out?

Unsafe bypasses many of the safety guarantees normally provided by Java.

Traditional Java:

```
Java code
↓
JVM safety checks
↓
Managed memory
↓
Garbage collector
```

Unsafe can allow:

```
Java code
↓
Unsafe
↓
Low-level memory manipulation
↓
Potentially outside normal JVM safety guarantees
```

Incorrect use can cause:

- JVM crashes
- memory corruption
- invalid memory access
- difficult debugging
- portability problems
- compatibility problems with future JDKs

Oracle's documentation describes these unsupported memory-access methods as having standard replacements in `VarHandle` and the FFM API. ([Oracle Docs](#))

4. Advantages

Advantage	Explanation
Early migration warning	Developers discover Unsafe usage before it potentially becomes an error
Improves future compatibility	Encourages migration to supported APIs
Improves application safety	Reduces reliance on unsupported low-level APIs
Improves library maintainability	Libraries can move toward standard Java APIs
Protects JVM integrity	Discourages unsafe memory manipulation
Encourages supported APIs	VarHandle and FFM have standardized interfaces
Helps dependency migration	Identifies third-party libraries using deprecated functionality
Better long-term portability	Standard APIs are preferable to JDK internals
No immediate hard failure	Existing applications can continue while migration is planned
Supports ecosystem transition	Gives libraries time to remove Unsafe dependencies
Useful for JDK upgrades	Warnings reveal compatibility risks during migration
Improves security posture	Reduces unnecessary use of low-level unsupported operations

5. Disadvantages

Disadvantage	Explanation
Warnings can appear in production	Existing applications may suddenly produce warning messages after upgrading to Java 24

Third-party libraries may trigger them	Your own source code may not contain Unsafe
Can create log noise	Warnings may clutter stderr/logging systems
Migration can be difficult	Low-level libraries may rely heavily on Unsafe
Performance-sensitive code may need redesign	Replacing Unsafe requires careful benchmarking
Not every replacement is one-to-one	Some low-level techniques require architectural changes
Dependency upgrades may be required	The actual fix may belong to a third-party library
Temporary suppression can hide problems	Ignoring warnings delays migration
Native/off-heap code can be complex	FFM migration requires understanding memory segments and arenas
Library maintainers face compatibility work	Popular infrastructure libraries may need significant changes
Potential short-term performance differences	New implementation should be benchmarked
Legacy applications may be difficult to modernize	Especially applications built around internal JVM assumptions

6. Traditional Unsafe Usage

A simplified example:

```
import sun.misc.Unsafe;

class UnsafeExample {

    static Unsafe unsafe;

    static {
        // Obtaining Unsafe is intentionally omitted here.
    }

    int readMemory(long address) {
        return unsafe.getInt(address);
    }
}
```

```
}  
}
```

This kind of direct memory access is precisely the type of functionality Java is moving away from.

The problem isn't simply:

"Unsafe is old"

It is:

Unsupported internal API
+
Low-level memory manipulation
+
JVM integrity risk
+
Future compatibility risk

7. Recommended Replacement — VarHandle

For many **memory-access and concurrency** use cases, VarHandle is the preferred replacement.

For example:

```
import java.lang.invoke.MethodHandles;  
import java.lang.invoke.VarHandle;  
  
class Counter {  
  
    private volatile int value;  
  
    private static final VarHandle VALUE;  
  
    static {  
        try {  
            VALUE = MethodHandles.lookup()  
                .findVarHandle(  
                    Counter.class,  
                    "value",  
                    int.class  
                );  
        } catch (ReflectiveOperationException e) {  
            throw new ExceptionInInitializerError(e);  
        }  
    }  
}
```

```
void increment() {  
    VALUE.getAndAdd(this, 1);  
}
```

Instead of:

Unsafe
↓
direct field manipulation

you have:

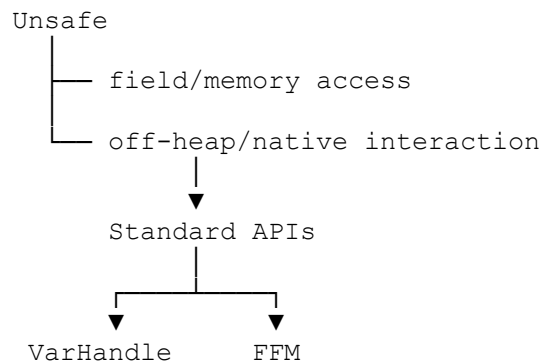
VarHandle
↓
standardized JVM-supported access

The Java platform documentation identifies VarHandle as one of the standard replacements for the deprecated Unsafe memory-access methods. ([Oracle Docs](#))

8. Recommended Replacement — Foreign Function & Memory API

For **off-heap memory and native interoperability**, the Foreign Function & Memory API is generally the more appropriate replacement.

Conceptually:



The FFM API became final in Java 22 and is designed to provide controlled access to foreign memory and foreign functions.

Oracle explicitly recommends the FFM API as a replacement for relevant Unsafe memory-access use cases. ([Oracle Docs](#))

9. When to Use the Feature

There is an important distinction:

You don't really "use" Unsafe Memory-Access Warnings. You use Java 24 and respond to the warnings.

Situation	Recommendation
Migrating Java 17/21 → 24	🔍 Very useful
Testing third-party dependencies	🔍 Very useful
Building infrastructure libraries	🔍 Pay close attention
Maintaining an old framework	🔍 Important
Running performance libraries	🔍 Important
CI compatibility testing	🔍 Excellent
Production migration	🔍 Monitor and remediate
New application development	🔍 Avoid new Unsafe usage
Writing low-level memory code	🔍 Prefer supported APIs
Simple business application	🔍 Usually no direct concern

10. When NOT to Use

Again, this is a warning mechanism rather than an API you deliberately call.

Situation	What to Do
Writing new application code	❌ Don't introduce Unsafe memory-access methods
Need atomic field operations	❌ Don't use Unsafe; use VarHandle/atomics
Need off-heap memory	❌ Prefer FFM

Need native function calls	✗ Prefer FFM
Need normal heap objects	✗ Use ordinary Java
Need concurrent counters	✗ Use AtomicInteger, AtomicLong, LongAdder, etc.
Need array access	✗ Prefer ordinary arrays/VarHandle where appropriate
Third-party library causes warning	✗ Don't blindly suppress it; upgrade or replace the library
Legacy code absolutely requires Unsafe temporarily	🔍 Isolate and create a migration plan

11. Real-Time Use Case — Atomic Counter

Old approach

Low-level libraries historically used Unsafe to perform atomic operations.

Conceptually:

```
unsafe.compareAndSwapInt(
    object,
    offset,
    expected,
    update
);
```

Modern approach

Use:

```
AtomicInteger counter =
    new AtomicInteger();

counter.incrementAndGet();
```

Or:

VarHandle

when more flexible memory-access modes are required.

Requirement	Preferred API
Simple atomic counter	AtomicInteger
Atomic long counter	AtomicLong
High-contention counter	LongAdder
Advanced field access/memory ordering	VarHandle
Native/off-heap memory	FFM

12. Real-Time Use Case — Concurrent Data Structure

Many high-performance collections historically used Unsafe internally.

For example, a library might use:

Unsafe
↓
CAS
↓
Lock-free queue

Modern implementation:

VarHandle
↓
CAS
↓
Lock-free queue

The application using the queue doesn't necessarily change.

This is important:

The migration often belongs inside the library implementation rather than application code.

13. Real-Time Use Case — Off-Heap Memory

Legacy implementation:

Application
↓
Unsafe.allocateMemory()
↓
Native/off-heap memory

Modern approach:

Application
↓
FFM API
↓
MemorySegment
↓
Foreign memory

Conceptually:

```
try (Arena arena = Arena.ofConfined()) {  
  
    MemorySegment segment =  
        arena.allocate(1024);  
  
    // use segment  
}
```

This provides a structured lifetime for foreign memory.

Compared with raw Unsafe memory management, the FFM model gives the application explicit memory-lifetime and access mechanisms.

14. Real-Time Use Case — Serialization Framework

A serialization framework might historically use Unsafe to:

- allocate objects without normal constructors
- access fields directly
- read/write memory efficiently
- bypass normal access rules

Architecture:

Old:
Serializer

↓
Unsafe
↓
Object fields

Modern migration might use:

Serializer
↓
VarHandle / reflection / MethodHandle
↓
Object fields

or redesign object construction around supported Java mechanisms.

This is one reason upgrading third-party libraries is important when moving to Java 24.

Oracle recommends checking and updating third-party libraries as part of migration. ([Oracle Docs](#))

15. Real-Time Use Case — High-Performance Networking

Networking frameworks historically used Unsafe for:

- direct buffers
- memory offsets
- atomic operations
- low-level array access

Architecture:

Legacy:
Network Framework
↓
Unsafe
↓
Direct memory

Modern architecture:

Network Framework
↓
VarHandle / FFM / supported JDK APIs
↓
Managed or foreign memory

The actual replacement depends on what the library is doing.

16. Real-Time Use Case — Memory-Mapped Data

A legacy system might use low-level mechanisms to manipulate mapped/off-heap memory.

Modern architecture could use:

Mapped/foreign memory



MemorySegment / FFM



VarHandle



Typed access

For example:

MemorySegment



ValueLayout



VarHandle-like access



Primitive value

This is much more explicit than arbitrary raw memory manipulation.

17. Real-Time Use Case — JVM Libraries

Libraries such as:

- high-performance collections
- serialization libraries
- networking libraries
- caching libraries
- concurrency libraries
- database drivers

may contain Unsafe dependencies.

The migration workflow should be:

Upgrade to Java 24



Run tests



```
Observe Unsafe warnings
  ↓
Identify dependency
  ↓
Upgrade library
  ↓
If no upgrade exists:
  evaluate migration/fork/replacement
```

18. Finding the Problematic Dependency

If the warning appears but you don't know which library caused it, investigate the stack trace and dependency tree.

For example:

```
Application
  ↓
Library A
  ↓
Library B
  ↓
sun.misc.Unsafe
```

Your application may contain **zero direct Unsafe references**.

This is common in mature infrastructure libraries.

Oracle recommends updating third-party libraries as part of JDK migration and using dependency-analysis tooling such as `jdeps` to identify problematic dependencies. ([Oracle Docs](#))

19. Unsafe vs VarHandle

Aspect	sun.misc.Unsafe	VarHandle
API type	Internal/unsupported	Standard Java API
Package	sun.misc	java.lang.invoke
Supported	✗	✓
Future compatibility	Poor	Good
Atomic operations	✓	✓

Memory ordering	Low-level	Explicit modes
Field access	✓	✓
Array access	✓	✓
CAS	✓	✓
Low-level flexibility	Very high	High
Safety	Lower	Better
Recommended for new code	✗	?

20. Unsafe vs FFM

Aspect	sun.misc.Unsafe	FFM API
Package	sun.misc	java.lang.foreign
Supported	✗	✓
Off-heap memory	✓	✓
Native functions	Limited/indirect	✓
Memory lifetime	Manual/dangerous	Explicit scopes/arenas
Memory safety	Low	Better
Native interoperability	Low-level	First-class
Future direction	✗	?
Recommended	✗	?

21. Unsafe vs Atomic Classes

For ordinary concurrency:

Requirement	Unsafe	Atomic Classes
Atomic integer	Possible	★★★★★
Atomic long	Possible	★★★★★
CAS	Possible	★★★★★
Simple counter	Overkill	★★★★★
Maintainability	Poor	Excellent
Supported API	✗	✓
Beginner friendly	✗	✓

Example:

```
AtomicLong counter =
    new AtomicLong();
```

```
counter.incrementAndGet();
```

is preferable to implementing a counter using raw Unsafe.

22. Alternate Features

Alternative	Main Purpose	Best Replacement For
VarHandle	Low-level heap/field/array access	Unsafe memory-access methods
Foreign Function & Memory API	Foreign/off-heap memory and native calls	Unsafe native/off-heap use
AtomicInteger/AtomicLong	Atomic state	CAS-based counters
LongAdder	High-throughput counters	Contended counters
MethodHandle	Dynamic invocation	Some low-level reflective mechanisms
ByteBuffer	Buffer-based memory access	Some direct-memory use cases

Standard Java objects/arrays	Normal memory	Unnecessary Unsafe usage
JNI	Native interoperability	Only when FFM doesn't fit or compatibility requires it

23. Which Alternative Should You Choose?

Requirement	Best Alternative
Atomic integer	AtomicInteger
Atomic long	AtomicLong
High-contention counter	LongAdder
CAS on object field	VarHandle
Memory ordering	VarHandle
Heap field access	VarHandle
Array element access	VarHandle / normal arrays
Off-heap memory	FFM API
Native function invocation	FFM API
Normal Java object allocation	new
Normal arrays	Java arrays
Simple byte storage	byte[] / ByteBuffer
Legacy native integration	JNI, preferably with migration assessment

24. Performance Considerations

A common misconception is:

"Unsafe is always faster than supported APIs."

That should **not** be assumed.

Area	Consideration
Raw Unsafe	Can be extremely low-level
VarHandle	Designed for efficient low-level access
Atomic classes	Highly optimized for common patterns
FFM	Designed for efficient foreign-memory/native interaction
JIT optimization	Can optimize supported APIs aggressively
Benchmarking	Essential for performance-sensitive migration
Safety	Supported APIs provide better guarantees
Future compatibility	Supported APIs win

The correct approach is:

Unsafe implementation
↓
Supported API replacement
↓
Benchmark
↓
Profile
↓
Optimize if necessary

Don't preserve Unsafe merely because someone believes it is faster.

25. Security Considerations

Risk	Unsafe	Supported API
Memory corruption risk	⚠ Higher	⚠ Lower
JVM integrity risk	⚠ Higher	⚠ Lower

Unsupported API dependency	☐ Yes	☐ No
Future compatibility	☐ Poor	☐ Better
Controlled memory lifetime	☐	☐ FFM
API stability	☐	☐
Maintainability	☐	☐

Oracle's restricted-method documentation notes that low-level APIs can be capable of violating JVM integrity when used incorrectly. ([Oracle Docs](#))

26. Migration Strategy

Step	Action
1	Upgrade application to Java 24
2	Run the application and tests
3	Capture Unsafe warnings
4	Identify the caller/library
5	Check for a newer library version
6	Upgrade dependency
7	If it's your code, replace Unsafe
8	Use VarHandle for heap/memory access
9	Use FFM for foreign/off-heap memory
10	Use atomic classes for ordinary concurrency
11	Benchmark
12	Test on future JDK versions
13	Remove temporary compatibility workarounds

27. Migration Decision Tree

Does your code/library use `sun.misc.Unsafe`?

Yes

What does it use `Unsafe` for?

Field/
array
access

Atomic
operation

Off-heap/
native memory

VarHandle

Atomic
classes

FFM API

This is a useful practical rule.

28. What About `Unsafe.allocateInstance()`?

This is an important distinction.

JEP 498 specifically targets **memory-access methods**.

Not every `Unsafe` method is treated identically by JEP 498.

For example:

```
unsafe.allocateInstance(MyClass.class);
```

is conceptually different from:

```
unsafe.getInt(object, offset);
```

However, `Unsafe` as a whole remains an unsupported internal API, and many of its methods have been deprecated for removal or have supported alternatives.

Therefore:

Don't interpret the absence of a JEP 498 warning as proof that a particular Unsafe method is safe for long-term use.

29. Important Difference: Warning vs Error

Java 24 behavior:

Unsafe memory-access method



Invocation



Warning



Operation continues

This is intentionally a migration phase.

The strategic direction is:

JDK 23



Deprecated for removal



JDK 24



Runtime warning



Future JDK



Potential removal/restriction

Oracle's migration guidance recommends assuming these methods will eventually disappear and moving to supported APIs. ([Oracle Docs](#))

30. Production Decision Table

Situation	Recommendation
New application	❗ Never introduce Unsafe memory-access methods
Existing application warning	❗ Investigate
Third-party dependency warning	❗ Upgrade dependency

Own library uses Unsafe	🔗 Migrate
Atomic operation	🔗 VarHandle/Atomic classes
Off-heap memory	🔗 FFM
Native function	🔗 FFM
Temporary legacy dependency	🔗 Isolate and plan migration
No warning	🔗 Still review dependencies
Performance-sensitive replacement	🔗 Benchmark before/after
Production suppression	🔗 Avoid hiding warnings indefinitely

31. Best Practices

Practice	Recommendation
Avoid new Unsafe code	🔗 Strongly recommended
Replace field access with VarHandle	🔗
Replace off-heap memory with FFM	🔗
Replace simple atomics with atomic classes	🔗
Upgrade third-party dependencies	🔗
Run tests on Java 24	🔗
Treat warnings as migration tasks	🔗
Benchmark low-level replacements	🔗
Don't blindly suppress warnings	🔗
Don't assume Unsafe is automatically faster	🔗
Keep low-level code isolated	🔗

Follow future JDK migration guidance	?
---	---

32. Overall Advantages vs Disadvantages

Category	Assessment
Migration visibility	? Excellent
Future compatibility	? Excellent
JVM safety	? Excellent
Developer awareness	? Excellent
Third-party dependency detection	? Excellent
Immediate application benefit	? Moderate
Migration effort	? Medium/High
Legacy application compatibility	?
Warning/log noise	?
Performance migration	? Requires benchmarking
Long-term maintainability	? Excellent
New application usefulness	? Avoidance guidance

33. Final Recommendation

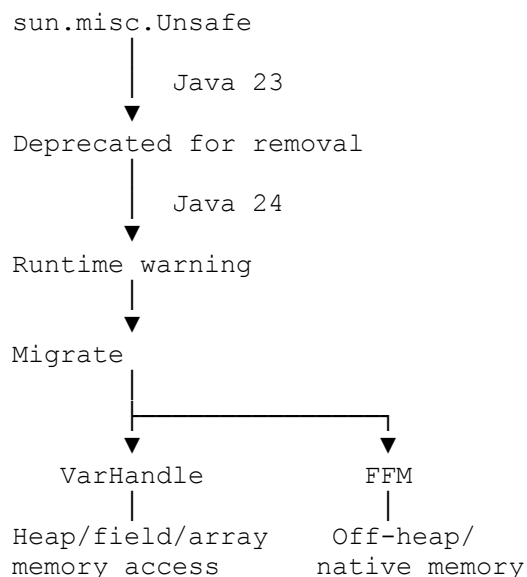
Requirement	Best Choice
Low-level heap field access	VarHandle
Atomic field operations	VarHandle / Atomic classes
Atomic counter	AtomicInteger / AtomicLong

High-contention counter	LongAdder
Off-heap memory	FFM API
Native functions	FFM API
Normal application memory	Ordinary Java objects/arrays
Existing Unsafe code	Migrate
Third-party Unsafe warning	Upgrade dependency
New Java application	Do not introduce Unsafe memory-access methods

Bottom line

Unsafe Memory-Access Warnings are primarily a migration feature, not a feature you deliberately add to your application.

The Java 24 message is essentially:



The most important mapping is:

Old Unsafe usage	Preferred modern API
getInt / putInt on fields	VarHandle
compareAndSwap...	VarHandle / Atomic classes

Atomic counters	AtomicInteger / AtomicLong / LongAdder
Direct/off-heap memory	Foreign Function & Memory API
Native functions	Foreign Function & Memory API
Normal object access	Ordinary Java

So, for a Java 24 migration, **do not simply suppress the warning**. Treat it as a dependency/code-modernization signal. First identify who is calling Unsafe, then upgrade the library or migrate your own implementation to **VarHandle, FFM, or the appropriate high-level concurrency API**. Oracle explicitly recommends this migration path. ([Oracle Docs](#))

Java 24 — Structured Concurrency (Fourth Preview)

Property	Details
Java Version	24
Year	2025
Feature	Structured Concurrency (Fourth Preview)
JEP	JEP 499
UniqueClassName	StructuredConcurrency
Status	Fourth Preview
First Preview	Java 19
Second Preview	Java 20
Third Preview	Java 21
Fourth Preview	Java 24
Primary API	java.util.concurrent.StructuredTaskScope
Main purpose	Treat related concurrent tasks as one structured unit of work
Works especially well with	Virtual threads

Core benefits	Clear task lifetime, cancellation, failure propagation, observability
Performance goal	Better scalable concurrency management, not automatic speedup
Production concern	Preview feature in Java 24

JEP 499 continues Structured Concurrency as a preview feature in Java 24. Its central idea is to make concurrent tasks follow the same lifetime and ownership structure as the code that starts them.

1. What Is Structured Concurrency?

Traditional concurrent code often looks like:

```
ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor();
```

```
Future<String> user =
    executor.submit(() -> getUser());
```

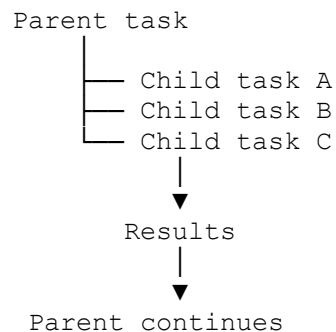
```
Future<List<Order>> orders =
    executor.submit(() -> getOrders());
```

```
String userResult = user.get();
```

```
List<Order> orderResult = orders.get();
```

The tasks are launched separately from the code that manages them.

Structured concurrency tries to make the relationship explicit:



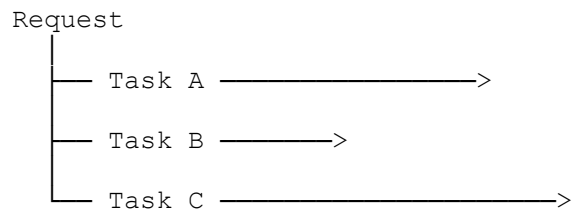
The parent task owns the child tasks.

The key principle is:

Concurrent tasks should have a clear lexical lifetime and ownership relationship.

2. The Core Idea

Without structured concurrency:



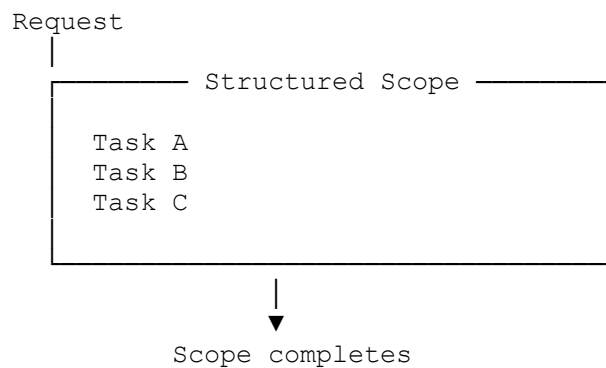
Request may finish
while child tasks are still running

Request may finish
while child tasks are still running

This can lead to:

- leaked tasks
- forgotten cancellation
- difficult exception handling
- unclear task ownership
- difficult debugging

With structured concurrency:



The scope doesn't finish until its child tasks have been handled.

3. Basic Example

A simplified Java 24 example:

```
try (var scope = StructuredTaskScope.open()) {  
  
    var user =  
        scope.fork(() -> findUser());  
  
    var orders =  
        scope.fork(() -> findOrders());  
  
    scope.join();  
  
    String userResult = user.get();  
    List<Order> orderResult = orders.get();  
  
    return new Dashboard(  
        userResult,  
        orderResult  
    );  
}
```

The important relationship is:

```
scope  
├─ user task  
└─ orders task  
    ↓  
  join()  
    ↓  
retrieve results  
    ↓  
scope closes
```

The exact API surface in preview releases can evolve, so Java 24 code using JEP 499 should be compiled/run with preview support and checked against the Java 24 API documentation.

4. Why Is It Useful?

Imagine a REST endpoint:

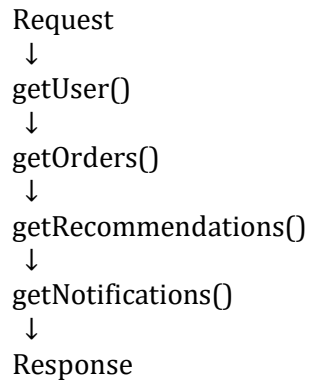
GET /dashboard

It needs:

User information

Orders
Recommendations
Notifications

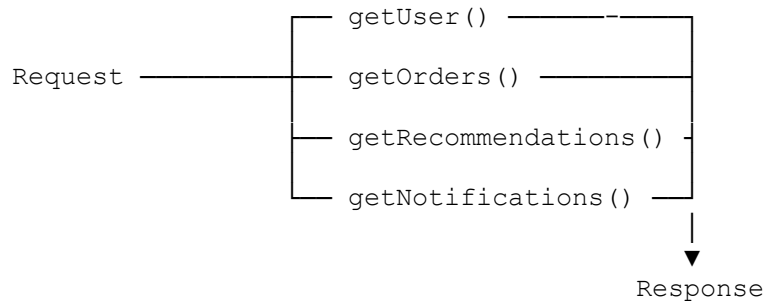
Sequential implementation:



If each takes 100 ms:

~400 ms

Parallel implementation:



The idealized latency approaches the slowest task rather than the sum of all task durations, subject to real-world constraints such as network latency, service capacity, CPU, and contention.

Structured concurrency provides a cleaner way to express this parallelism.

5. Advantages

Advantage	Explanation
Clear task ownership	Parent scope owns child tasks
Clear task lifetime	Child tasks normally finish before the scope

	completes
Automatic cleanup model	Scope closure helps prevent abandoned tasks
Cancellation propagation	Failure/cancellation can be propagated to sibling tasks
Failure handling is easier	Related task failures can be treated as one operation
Better observability	Task relationships are easier to understand
Works naturally with virtual threads	Lightweight child tasks make concurrent decomposition practical
Simplifies parallel I/O	Excellent for calling independent services concurrently
Reduces Future management	Less manual coordination
Avoids orphan tasks	Scope gives tasks a bounded lifetime
Improves readability	Concurrent code follows program structure
Encourages structured design	Concurrency resembles normal call hierarchy
Useful for request processing	Request → child tasks → response
Useful for fan-out/fan-in	Launch many tasks and combine results
Better error propagation	Failure can trigger sibling cancellation
Supports deadlines/timeouts	Useful for bounded operations
Can simplify nested concurrency	Child scopes can represent nested work

6. Disadvantages

Disadvantage	Explanation
Still a preview feature	Java 24 requires preview support
API may change	Preview APIs aren't guaranteed to remain identical
Learning curve	Developers must learn structured-concurrency

	concepts
Not appropriate for every task	Independent long-lived background work may need another model
Requires careful cancellation design	Tasks must respond appropriately to interruption/cancellation
Failure policy needs thought	Different tasks may have different importance
Potentially more code for simple operations	Parallelizing trivial work can add unnecessary complexity
Not a magic performance feature	Parallel execution doesn't automatically make applications faster
Resource limits still matter	Parallel tasks can overload downstream services
CPU-bound work still consumes CPU	More tasks don't create more CPU capacity
Nested scopes can become complex	Poorly designed scopes can reduce readability
Preview tooling considerations	Build/deployment pipelines must support preview compilation

7. Structured Concurrency vs Traditional Threads

Aspect	Traditional Threads	Structured Concurrency
Task ownership	Often implicit	Explicit
Task lifetime	Developer-managed	Scope-managed
Cancellation	Manual	Structured
Error propagation	Manual	Structured
Child relationship	Often unclear	Explicit
Resource cleanup	Manual	Scope-oriented
Observability	Harder	Better
Virtual thread integration	Possible	Excellent

Code readability	Medium	High
Long-lived background task	Good	Not primary use case

8. Structured Concurrency vs ExecutorService

Traditional:

```
ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor();
```

```
Future<User> user =
    executor.submit(this::getUser);
```

```
Future<List<Order>> orders =
    executor.submit(this::getOrders);
```

```
try {
    return new Dashboard(
        user.get(),
        orders.get()
    );
} finally {
    executor.shutdown();
}
```

Structured approach:

```
try (var scope = StructuredTaskScope.open()) {

    var user =
        scope.fork(this::getUser);

    var orders =
        scope.fork(this::getOrders);

    scope.join();

    return new Dashboard(
        user.get(),
        orders.get()
    );
}
```


Concern	ExecutorService	Structured Concurrency
Task submission	✓	✓
Parallel execution	✓	✓
Virtual threads	✓	✓
Task ownership	?	?
Lifetime management	?	?
Cancellation	Manual	Structured
Failure propagation	Manual	Structured
Scope semantics	✗	✓
Best for request-scoped parallel work	Good	Excellent
Best for long-lived workers	Excellent	Not primary

9. When to Use

Situation	Recommendation	Reason
Parallel REST API calls	✓ Excellent	Natural fan-out/fan-in
Microservice aggregation	✓ Excellent	Independent downstream calls
Database calls in parallel	✓ Good	When independent and pool capacity allows
File operations	✓ Good	Independent I/O
Multiple remote API calls	✓ Excellent	Reduce aggregate latency
Search across multiple systems	✓ Excellent	Parallel search
Dashboard aggregation	✓ Excellent	Multiple independent data sources
Request-scoped tasks	✓ Excellent	Clear task lifetime

Fail-fast operations	❑ Excellent	Cancel siblings on failure
Timeout-sensitive requests	❑ Excellent	Scope can represent operation lifetime
Fan-out/fan-in processing	❑ Excellent	Natural model
Virtual-thread applications	❑ Excellent	Strong combination
Small sequential operation	❑ Unnecessary	No concurrency benefit
Long-lived background worker	❑ Usually not	Different lifecycle
CPU-heavy computation	❑ Carefully	CPU capacity remains limited

10. When NOT to Use

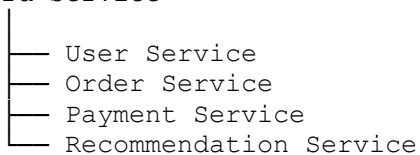
Situation	Why Avoid It?	Better Alternative
Simple sequential code	Adds unnecessary complexity	Normal method calls
One trivial task	No concurrency benefit	Direct call
Long-lived daemon task	Lifecycle doesn't match request scope	Executor/service
Scheduled recurring jobs	Structured scope isn't a scheduler	ScheduledExecutorService
Message consumer running indefinitely	Not naturally request-scoped	Executor/container
CPU-bound computation	More concurrency may not help	ForkJoinPool/parallelism
Shared background service	Scope ownership differs	Executor/service lifecycle
Existing simple ExecutorService design	Migration may provide little value	Keep existing design

API requires asynchronous completion beyond parent lifetime	Structured lifetime may be inappropriate	CompletableFuture/executor
Massive unbounded fan-out	Can overload resources	Bounded concurrency

11. Real-Time Use Case — Microservice Aggregator

Suppose:

Dashboard Service



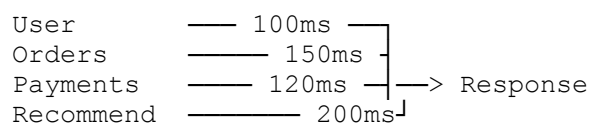
Sequential:

```

User      100ms
Orders    150ms
Payments  120ms
Recommend 200ms
  
```

Total $\approx$ 570ms

Parallel:



Idealized critical path:

$\approx$ 200ms

plus coordination and other overhead.

Structured concurrency expresses this naturally:

```

try (var scope = StructuredTaskScope.open()) {

    var user =
  
```

```

    scope.fork(userService::getUser);

var orders =
    scope.fork(orderService::getOrders);

var payments =
    scope.fork(paymentService::getPayments);

var recommendations =
    scope.fork(recommendationService::getRecommendations);

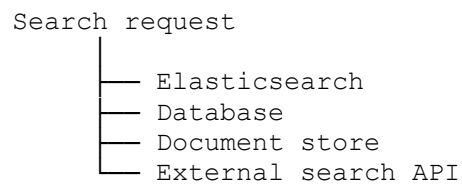
scope.join();

return new Dashboard(
    user.get(),
    orders.get(),
    payments.get(),
    recommendations.get()
);
}

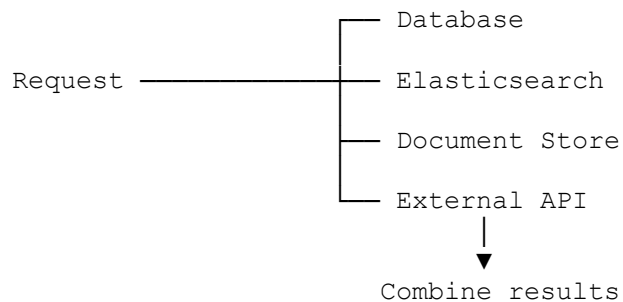
```

12. Real-Time Use Case — Search Aggregator

Imagine an enterprise search service:



All searches can happen concurrently.



Structured concurrency is a natural fit because all searches belong to one request.

If the request ends:

Request scope ends



Child search tasks end

You don't want search tasks continuing indefinitely after the client has disconnected.

13. Real-Time Use Case — Fail-Fast Operations

Suppose a request requires:

```
Inventory
+
Pricing
+
Fraud check
```

If fraud validation fails:

```
Fraud check
  ↓
FAIL
  ↓
Cancel inventory task
Cancel pricing task
  ↓
Reject request
```

Structured concurrency is especially useful when sibling tasks are meaningful only if the overall operation succeeds.

Conceptually:

```
try (var scope = StructuredTaskScope.open()) {

    var inventory =
        scope.fork(this::checkInventory);

    var pricing =
        scope.fork(this::calculatePrice);

    var fraud =
        scope.fork(this::checkFraud);

    scope.join();

    // evaluate results
}
```

The exact shutdown/failure policy depends on the structured task scope configuration.

14. Real-Time Use Case — Timeout

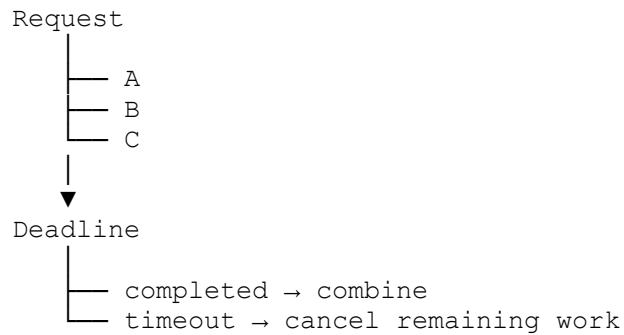
Suppose an API must respond within:

500 ms

You have:

Service A
Service B
Service C

Structured operation:



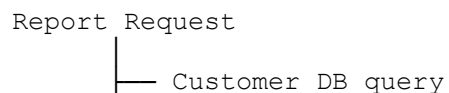
This is much easier to reason about than having several independent futures that might continue executing after the request has timed out.

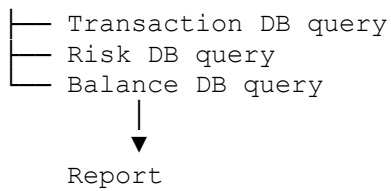
15. Real-Time Use Case — Parallel Database Queries

Suppose a reporting endpoint needs:

Customer information
Transaction summary
Risk profile
Account balances

These are independent queries.





Structured concurrency can organize those queries under one operation.

Important: don't blindly increase database concurrency. The database connection pool remains a resource limit.

16. Real-Time Use Case — Multiple HTTP APIs

For example:

```
try (var scope = StructuredTaskScope.open()) {  
  
    var weather =  
        scope.fork(() -> weatherApi.get());  
  
    var traffic =  
        scope.fork(() -> trafficApi.get());  
  
    var maps =  
        scope.fork(() -> mapApi.get());  
  
    scope.join();  
  
    return new TravelInfo(  
        weather.get(),  
        traffic.get(),  
        maps.get()  
    );  
}
```

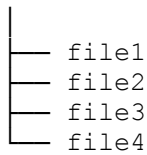
This is a strong use case because these are independent I/O-bound operations.

Virtual threads make it practical to have many such blocking operations concurrently.

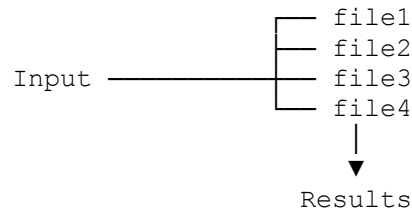
17. Real-Time Use Case — Parallel File Processing

Suppose:

Input directory



Each file can be processed independently.

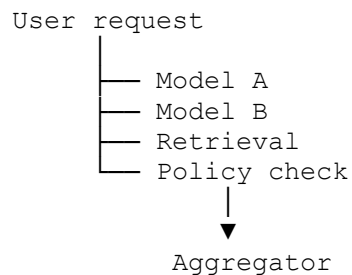


Structured concurrency can make the lifetime of these processing tasks explicit.

For large numbers of files, however, use **bounded concurrency** rather than creating an unbounded number of tasks.

18. Real-Time Use Case — AI/ML Service Aggregation

Consider an AI gateway:



The gateway can execute independent operations concurrently and combine their results.

Structured concurrency helps ensure:

One request
↓
all child work
↓
one lifecycle

If the request is cancelled:

Client disconnect
↓
Cancel child tasks
↓
Release resources

19. Real-Time Use Case — Parallel Validation

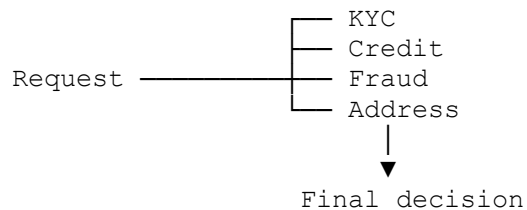
An enterprise onboarding request might require:

KYC check
Credit check
Fraud check
Address verification

Instead of:

KYC → Credit → Fraud → Address

you can potentially execute:



Structured concurrency is particularly appropriate because these tasks belong to the same business operation.

20. Structured Concurrency + Virtual Threads

These features complement each other.

Structured Concurrency

+

Virtual Threads

↓

Scalable structured I/O concurrency

Virtual threads answer:

How can I run many blocking tasks efficiently?

Structured concurrency answers:

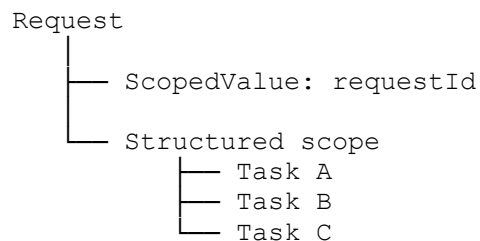
How should I organize, control, cancel, and observe those tasks?

Feature	Main Responsibility
Virtual threads	Lightweight execution
Structured concurrency	Task organization/lifecycle
Scoped values	Context propagation
ExecutorService	Task execution abstraction
CompletableFuture	Asynchronous composition

This combination is particularly useful in modern Java server applications.

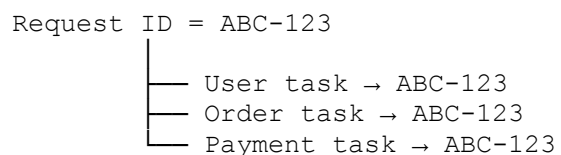
21. Structured Concurrency + Scoped Values

Another powerful Java concurrency combination is:



Child tasks can inherit the relevant scoped context.

For example:



This is useful for:

- Request tracing
- Tenant context

- Security context
- Correlation IDs
- Logging

The combination is particularly compelling for virtual-thread-based server applications.

22. Structured Concurrency vs CompletableFuture

Aspect	Structured Concurrency	CompletableFuture
Async composition	✓	★★★★★
Structured task lifetime	★★★★★	★★
Cancellation propagation	★★★★★	★★★
Request-scoped parallel work	★★★★★	★★★★
Complex async pipelines	★★★	★★★★★
Functional chaining	★★★	★★★★★
Error handling	★★★★★	★★★★
Readability for fan-out/fan-in	★★★★★	★★★
Long-lived async pipeline	★★★	★★★★★
Preview in Java 24	🔍	✗

23. Structured Concurrency vs Parallel Streams

Aspect	Structured Concurrency	Parallel Stream
Parallel I/O	★★★★★	★★
CPU data processing	★★★	★★★★★
Multiple independent services	★★★★★	★
Task cancellation	★★★★★	★★

Failure handling	★★★★★	★★★
Virtual threads	★★★★★	Not its purpose
Request-scoped tasks	★★★★★	★★
Collection transformations	★★★	★★★★★

Use parallel streams primarily for suitable **data-parallel computations**, not as a general replacement for structured concurrent I/O.

24. Structured Concurrency vs ExecutorService

Requirement	Best Choice
Request-scoped fan-out/fan-in	Structured Concurrency
Child task cancellation	Structured Concurrency
Clear task ownership	Structured Concurrency
Long-lived worker pool	ExecutorService
Scheduled execution	ScheduledExecutorService
Queue-based workers	ExecutorService
Simple task submission	Either
Existing mature executor architecture	Often keep ExecutorService
Virtual-thread per-task execution	Either, depending lifecycle needs

25. Structured Concurrency vs Reactive Programming

Aspect	Structured Concurrency	Reactive
Blocking I/O	★★★★★ with virtual threads	★★★
Non-blocking streams	★★★	★★★★★

Backpressure	☆☆	☆☆☆☆
Request aggregation	☆☆☆☆	☆☆☆☆
Readability	☆☆☆☆	☆☆
Complex event streams	☆☆	☆☆☆☆
Cancellation	☆☆☆☆	☆☆☆☆
Learning curve	Medium	High
Best use	Structured tasks	Continuous async streams

Reactive programming remains useful where **backpressure, continuous event streams, and asynchronous pipelines** are core requirements.

26. Alternate Features

There is no single direct replacement.

Alternative	Main Purpose	When Better
Virtual Threads	Lightweight concurrency	Need many concurrent blocking tasks
ExecutorService	Task execution	Long-lived worker pools
CompletableFuture	Async composition	Complex asynchronous pipelines
ForkJoinPool	CPU parallelism	Recursive/data-parallel computation
Parallel Streams	Data parallelism	Collection-oriented CPU workloads
Reactive Streams	Async pipelines/backpressure	Event-driven/high-throughput streams
ScheduledExecutorService	Scheduled work	Periodic/background tasks
Traditional Threads	Direct thread control	Special low-level requirements

27. Which Alternative Should You Choose?

Requirement	Recommended
Parallel HTTP calls within one request	Structured Concurrency
Parallel DB calls within one request	Structured Concurrency
Fan-out/fan-in	Structured Concurrency
Fail-fast sibling tasks	Structured Concurrency
Request cancellation	Structured Concurrency
Long-lived background workers	ExecutorService
Periodic tasks	ScheduledExecutorService
Complex async chaining	CompletableFuture
CPU-intensive parallel calculation	ForkJoinPool
Collection data processing	Parallel Stream
Backpressure/event streams	Reactive Streams
Maximum lightweight blocking concurrency	Virtual Threads

28. Common Mistake — Unbounded Fan-Out

Don't do:

```
for (Request request : requests) {  
    scope.fork(() -> process(request));  
}
```

if:

```
requests = 10,000,000
```

The fact that virtual threads are lightweight doesn't mean downstream systems have unlimited capacity.

Better architecture:

10 million requests
↓
Bounded concurrency
↓
Database/API capacity
↓
Controlled processing

Structured concurrency helps organize tasks, but **application-level concurrency limits are still necessary**.

29. Common Mistake — Using It for Background Jobs

Suppose:

HTTP request
↓
Start email job
↓
Return response immediately

If the email operation is intentionally independent of the request's lifecycle, making it a child of the request's structured scope may be inappropriate.

Instead:

```
HTTP request
  |
  └─ enqueue job
      ↓
    Background worker
```

Use:

- message queues
- executor/service lifecycle
- job scheduler
- event-driven architecture

depending on requirements.

30. Common Mistake — CPU-Bound Work

Suppose:

```
scope.fork() -> calculateHugePrimeNumber();  
scope.fork() -> calculateAnotherHugePrimeNumber();
```

Structured concurrency can organize the tasks, but it doesn't magically increase CPU capacity.

If you have:

8 CPU cores

and launch:

1000 CPU-heavy tasks

you still have approximately:

8 CPU cores

available.

For CPU-bound workloads, consider:

- ForkJoinPool
- bounded executor
- parallel algorithms
- vectorization
- batching

depending on the workload.

31. Failure Handling

One of structured concurrency's biggest benefits is failure management.

Traditional:

Task A fails

↓

Future A exception

Task B continues

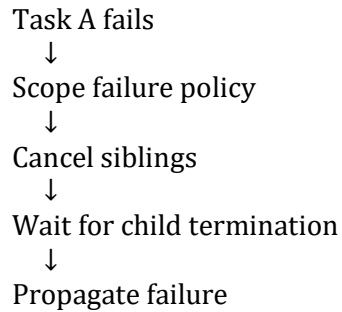
Task C continues

Potentially:

Parent request fails

but B and C are still running

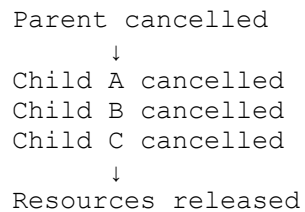
Structured model:



This makes the overall operation easier to reason about.

32. Cancellation Model

Good cancellation:



But cancellation isn't magic.

Your code must cooperate correctly with interruption/cancellation.

For example, blocking operations and libraries need to respond appropriately.

33. Performance Considerations

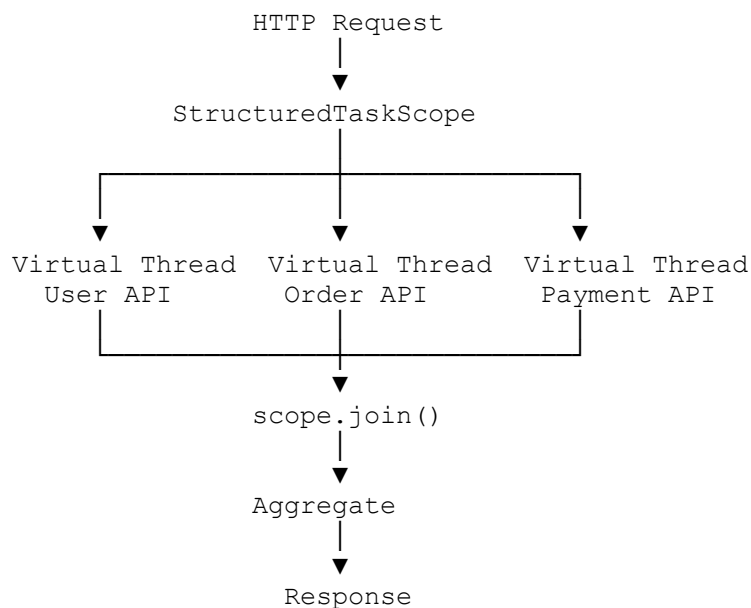
Area	Expected Impact
I/O concurrency	☑ Potentially excellent
Request latency	☑ Potentially lower for independent operations
CPU throughput	☑ Workload-dependent
Memory	☑ Virtual threads can be lightweight

Context switching	⌚ Often efficient with virtual threads
Network utilization	⌚ Potentially better
Downstream load	⌚ Can increase substantially
Database load	⌚ Must be controlled
Thread creation cost	⌚ Low with virtual threads
Overall application performance	⌚ Requires benchmarking

Structured concurrency itself is mainly a **correctness, lifecycle, and maintainability feature**.

34. Production Architecture Example

A modern Java service can look like:



This is one of the strongest real-world use cases.

35. Best Practices

Practice	Recommendation
----------	----------------

Use for request-scoped concurrent tasks	?
Use with virtual threads	?
Keep scopes small and meaningful	?
Give tasks clear responsibilities	?
Propagate cancellation	?
Define failure policy	?
Define timeouts/deadlines	?
Limit downstream concurrency	? Critical
Handle interruption correctly	? Critical
Avoid unbounded fan-out	? Critical
Don't use for permanent background jobs	?
Don't use concurrency without measurable benefit	?
Benchmark real workloads	?
Remember Java 24 preview status	?

36. Decision Table

Question	Recommendation
Do several tasks belong to one request?	? Structured Concurrency
Are tasks independent?	? Strong candidate
Do you need fan-out/fan-in?	? Excellent
Should failure cancel siblings?	? Excellent
Should cancellation propagate to children?	? Excellent
Are tasks mostly blocking I/O?	? Excellent with virtual threads

Is the work CPU-bound?	❓ Consider ForkJoinPool/bounded executor
Is the task intentionally long-lived?	❓ Use another lifecycle model
Is it periodic work?	❓ ScheduledExecutorService
Is it a continuous event stream?	❓ Reactive Streams
Is it a complex async pipeline?	❓ CompletableFuture
Is there only one task?	❓ Normal method call
Is there massive unbounded fan-out?	❓ Add bounded concurrency

37. Overall Advantages vs Disadvantages

Category	Assessment
Task ownership	❓ Excellent
Task lifetime	❓ Excellent
Cancellation	❓ Excellent
Failure propagation	❓ Excellent
Observability	❓ Excellent
Request aggregation	❓ Excellent
I/O concurrency	❓ Excellent
Virtual-thread integration	❓ Excellent
CPU-bound performance	❓ Depends
Background jobs	❓ Limited
Event streams	❓ Limited
Learning curve	❓ Moderate
API maturity in Java 24	❓ Preview
Enterprise usefulness	❓ Very high

38. Final Recommendation

Scenario	Best Choice
Parallel REST calls	★ Structured Concurrency
Microservice aggregation	★ Structured Concurrency
Parallel DB/API calls	★ Structured Concurrency
Fan-out/fan-in	★ Structured Concurrency
Fail-fast operations	★ Structured Concurrency
Request cancellation	★ Structured Concurrency
Virtual-thread applications	★ Structured Concurrency + Virtual Threads
Long-running worker	★ <code>ExecutorService</code>
Scheduled task	★ <code>ScheduledExecutorService</code>
Complex async pipeline	★ <code>CompletableFuture</code>
CPU-intensive parallelism	★ <code>ForkJoinPool</code>
Collection processing	★ Parallel Streams
Backpressure/event streams	★ Reactive Streams

Bottom line

Structured Concurrency is best understood as:

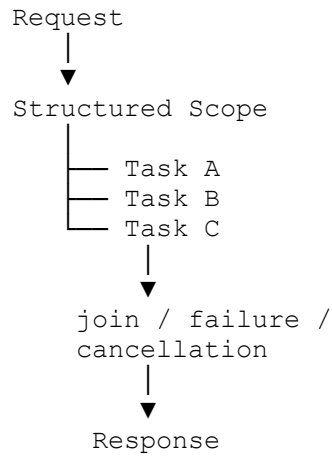
A way to make concurrent tasks behave like a structured extension of ordinary method calls.

Traditional:

```
Request
├── Future A
├── Future B
└── Future C
```

Who owns them?
When do they finish?
What happens if A fails?
Who cancels B and C?

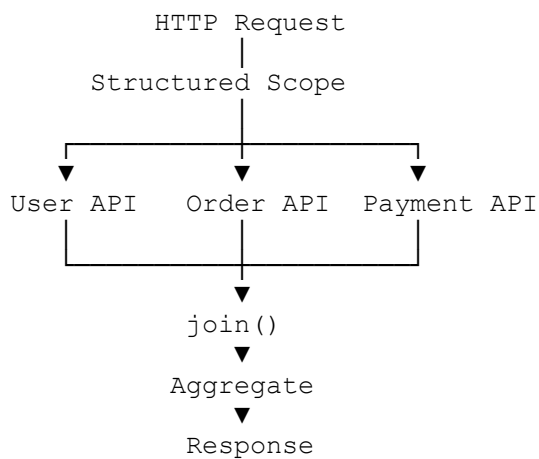
Structured:



The most powerful combination in modern Java is:

Virtual Threads
+
Structured Concurrency
+
Scoped Values
↓
Structured, scalable request concurrency

For example, a microservice request can execute:



Use Structured Concurrency when several short-lived concurrent tasks are logically part of one operation and should share a common lifetime, cancellation policy, and failure boundary. It is particularly compelling for I/O-heavy microservices using virtual threads.

Do not use it simply because "concurrency is faster." For long-lived background jobs, schedulers, continuous event streams, complex asynchronous pipelines, or CPU-bound workloads, `ExecutorService`, `ScheduledExecutorService`, `CompletableFuture`, reactive frameworks, or `ForkJoinPool` may be more appropriate.

And for your Java-version tracking: **JEP 499 was still a fourth-preview feature in Java 24**, so it should be treated as preview syntax/API rather than as a finalized Java feature.

Java 24 — Deprecate 32-bit x86 Port for Removal

Property	Details
Java Version	24
Year	2025
Feature	Deprecate 32-bit x86 Port for Removal
JEP	JEP 501
UniqueClassName	DeprecateX86Port
Type	Platform/JDK implementation change
Status in Java 24	Deprecated for removal
What it affects	32-bit x86 (x86_32) JDK port, primarily Linux 32-bit x86
What it does NOT affect	64-bit x86 (x86_64/AMD64), ARM64, other 64-bit platforms

JEP 501 deprecated the remaining **32-bit x86 port** with the intention of removing it in a future release. The motivation was that maintaining the 32-bit port increasingly consumed engineering effort needed for newer JVM capabilities such as Loom, FFM, Vector API, and newer GC/JIT work. ([OpenJDK Bug System](#))

Important: This is not a Java programming-language feature that application developers call from Java code. It is a **JDK/platform support decision**.

Also, this change was followed by **JEP 503: Remove the 32-bit x86 Port**, which was delivered in **Java 25**. ([OpenJDK Mail](#))

1. What problem does it solve?

Aspect	Explanation
Old platform	32-bit x86 has become a niche platform compared with 64-bit x86 and ARM64
Maintenance cost	JVM engineers had to maintain platform-specific code and tests for 32-bit x86
New JVM features	New capabilities increasingly require platform-specific implementation work
Engineering focus	Removing obsolete platform support lets OpenJDK concentrate on actively used architectures
Future development	Less legacy code means fewer architecture-specific fallbacks and less testing burden
Long-term direction	Java moves toward modern 64-bit platforms

The JEP specifically states that the cost of maintaining the port outweighed its benefits and that removing it would unblock new features from having to implement 32-bit x86 fallbacks. ([OpenJDK Bug System](#))

2. Advantages

Advantage	Explanation	Real-world benefit
1. Reduced JDK maintenance	OpenJDK no longer needs to maintain as much 32-bit-specific implementation code	Faster JVM development
2. Simpler JVM implementation	Fewer architecture-specific branches and compatibility paths	Easier HotSpot maintenance
3. Better focus on modern platforms	Engineering effort can concentrate on x86-64, AArch64, etc.	Better support for current hardware
4. Easier implementation of	New features don't need to	Faster adoption of JVM

new JVM features	preserve 32-bit x86 compatibility	improvements
5. Reduced testing burden	Fewer platform combinations need continuous testing	More efficient OpenJDK testing
6. Better alignment with modern hardware	Modern servers, desktops and cloud systems are overwhelmingly 64-bit	More realistic platform priorities
7. Encourages modernization	Organizations running old 32-bit systems are encouraged to migrate	Better long-term maintainability
8. Helps remove legacy code	Old architecture-specific code can eventually disappear	Smaller and cleaner JVM implementation
9. Supports future JVM innovation	Features such as newer GC, Loom and FFM don't need to preserve obsolete platform paths	More freedom for JVM evolution
10. Reduced downstream maintenance	Vendors maintaining their own JDK builds can also eliminate obsolete x86_32 work	Simpler custom JDK distributions

The JEP explicitly cites maintenance parity with **Loom**, **FFM**, **Vector API**, and **GC barrier work** as examples of the opportunity cost of continuing to support 32-bit x86. ([OpenJDK Bug System](#))

3. Disadvantages

Disadvantage	Explanation	Impact
1. Legacy systems lose native JDK support	Applications running on 32-bit x86 cannot continue indefinitely with normal native JDK support	Migration required
2. Hardware upgrade may be required	Old 32-bit machines may need replacement	Infrastructure cost
3. OS migration may be required	A 32-bit Linux environment may need to	Migration effort

	move to 64-bit Linux	
4. Native libraries can complicate migration	JNI libraries compiled specifically for 32-bit x86 may not work on 64-bit systems	Native dependency rebuilds
5. Embedded/industrial systems may be affected	Some older appliances and industrial devices remain 32-bit	Potential lifecycle issues
6. Vendor-specific JDKs may differ	Some vendors may have different support policies	Deployment compatibility must be checked
7. Old build pipelines may fail	CI/CD systems targeting x86_32 may no longer produce supported JDK builds	Build changes required
8. Legacy application testing increases	Organizations must validate the application on a supported architecture	Migration/testing cost
9. Specialized hardware can be difficult to replace	Some old equipment cannot easily be upgraded	Operational risk
10. Java application itself may not be the problem	The application source can be perfectly portable while the platform is obsolete	Infrastructure, not source-code, migration

4. When should you use it?

Strictly speaking, **you don't "use" JEP 501**. It changes which platform you should target.

Situation	Recommendation	Why
New enterprise application	✓ Use a 64-bit JDK	Avoid building on a deprecated architecture
New microservice	✓ Use x86-64 or ARM64	Modern cloud/container environments
New Docker deployment	✓ Use a 64-bit base image	Better ecosystem support

New Java server	✓ Use 64-bit OS/JDK	Modern JVM workloads expect this
Cloud deployment	✓ Use 64-bit	Cloud infrastructure is predominantly 64-bit
New embedded system	✓ Prefer 64-bit if hardware permits	Better long-term JDK compatibility
Long-lived enterprise product	✓ Migrate away from x86_32	Future Java compatibility
Existing 32-bit application	🔍 Plan migration	Java 24 is a warning that the platform is approaching removal
Legacy 32-bit device	🔍 Evaluate carefully	Hardware replacement may be difficult
32-bit Linux production system	🔍 Migrate to 64-bit	The remaining x86_32 JDK port was deprecated for removal

Oracle's migration guidance recommends checking supported system configurations and testing applications on the newer JDK during migration. ([Oracle Docs](#))

5. When should you NOT use it?

Since JEP 501 isn't an API, this really means **when should you not target 32-bit x86?**

Situation	Recommendation	Reason
New production application	✗ Don't target 32-bit x86	It is on the removal path
New cloud application	✗ Don't target 32-bit x86	64-bit platforms are the practical standard
New Kubernetes deployment	✗ Don't target 32-bit x86	Container ecosystem is overwhelmingly 64-bit
High-memory application	✗ Don't target 32-bit x86	32-bit address space is inherently restrictive
High-performance application	✗ Don't target 32-bit x86	Modern JVM optimizations target current architectures
New AI/ML application	✗ Don't target 32-bit x86	Modern hardware/software stacks are 64-bit

Application requiring current JVM capabilities	✗ Don't build around x86_32	Platform support is being removed
New enterprise infrastructure	✗ Don't invest in x86_32	Creates future migration debt
Existing irreplaceable legacy hardware	⌚ Don't immediately remove it	First establish a migration/upgrade strategy
Temporary legacy deployment	⌚ May remain temporarily	But treat it as technical debt

6. Real-time use cases

Use Case 1 — Enterprise application migration

Old architecture

Java Application
 ↓
 32-bit Linux
 ↓
 32-bit x86 CPU
 ↓
 Old Server

Recommended architecture

Java Application
 ↓
 64-bit Linux
 ↓
 x86-64 / ARM64 CPU
 ↓
 Modern Server / Cloud

Aspect	Old	Recommended
CPU	x86 32-bit	x86-64 / ARM64
OS	32-bit	64-bit
JDK	32-bit JDK	64-bit JDK
Future Java support	Poor	Strong

Migration direction	×	✓

7. Real-time use case — Docker

Suppose an organization has an old 32-bit Java application.

Old deployment

```
FROM old-32bit-java
COPY app.jar /app/app.jar
CMD ["java", "-jar", "/app/app.jar"]
```

Instead, migrate to a supported 64-bit environment:

```
FROM eclipse-temurin:24-jre
COPY app.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

The important migration is not a Java-code change. It is:

```
32-bit OS/JDK
  ↓
64-bit OS/JDK
```

8. Real-time use case — JNI application

This is one of the areas that requires particular care.

Suppose Java calls a native library:

```
public class PaymentTerminal {

    static {
        System.loadLibrary("payment");
    }

    public native int processPayment(byte[] request);
}
```

If the native library was compiled for 32-bit x86:

Java application



JNI



libpayment.so ← 32-bit x86

Simply moving Java to 64-bit isn't enough.

You need:

Java application



JNI



libpayment.so ← 64-bit x86-64

Therefore:

Component	Migration requirement
Java source	Usually little/no change
JDK	32-bit → 64-bit
OS	32-bit → 64-bit
JNI library	Recompile for 64-bit
Native dependencies	Verify/rebuild
CI/CD	Add 64-bit build
Testing	Full integration test

9. Real-time use case — Industrial/embedded Java system

Imagine an old industrial monitoring application:

Industrial sensor



32-bit embedded Linux



32-bit x86 CPU



Java monitoring application

JEP 501 means this architecture should be treated as a **migration candidate**, rather than something to build new infrastructure around.

A future architecture could be:

Industrial sensor
↓
64-bit Linux
↓
ARM64 / x86-64
↓
Modern Java

This is particularly important for systems with a 10–15 year operational lifetime.

10. Real-time use case — Java server with large memory

A 32-bit JVM is fundamentally constrained by its address space.

For example:

32-bit JVM
↓
Limited address space
↓
Limited practical heap
↓
Poor fit for modern large-memory servers

A 64-bit JVM provides the appropriate address-space model for modern server workloads.

Therefore:

-Xmx8g

or larger server heaps are naturally associated with 64-bit JVM deployments rather than legacy 32-bit environments.

11. Does this change Java source code?

Usually **no**.

For normal Java:

```
public class OrderService {

    public void processOrder() {
        System.out.println("Processing order");
    }
}
```

there is generally no:

```
if (is32BitX86) {
    ...
}
```

required.

The main migration concerns are:

Area	Concern
Java source	Usually low
JDK distribution	High
OS	High
Native libraries	Potentially high
JNI	Potentially high
Build scripts	Medium
Deployment	Medium/high
Docker images	Medium
Hardware	Potentially high

12. Alternate features / alternatives

There isn't a direct **Java language replacement** for JEP 501 because it is a platform-support change.

The practical alternatives are **supported target architectures**.

Alternative	Description	Best use
x86-64 / AMD64	64-bit successor to x86	Enterprise servers, desktops, cloud
AArch64 / ARM64	64-bit ARM architecture	Cloud, containers, modern servers, embedded
Zero VM	Architecture-agnostic interpreter	Specialized/unsupported architectures; not normally a performance replacement
64-bit OpenJDK	Supported modern JDK deployment	General enterprise Java
64-bit vendor JDK	Oracle/OpenJDK/vendor distribution	Production deployments
Native Image	Compiles Java applications to native executables	Specialized deployment scenarios, not a direct architecture replacement

JEP 501 specifically notes that after the 32-bit x86 port is removed, the **architecture-agnostic Zero port** is the remaining way to run Java programs on 32-bit x86 processors. However, Zero should not be viewed as equivalent to a normal optimized HotSpot port for production performance. ([OpenJDK Bug System](#))

13. 32-bit x86 vs x86-64 vs ARM64

Feature	x86 32-bit	x86-64	ARM64
Address space	Limited	Very large	Very large
Modern server suitability	✗ Poor	✓ Excellent	✓ Excellent
Cloud suitability	✗ Poor	✓ Excellent	✓ Excellent
Modern JVM direction	✗ Legacy	✓ Strong	✓ Strong
New application development	✗ Avoid	✓ Recommended	✓ Recommended
Long-term Java deployment	✗ Poor	✓ Excellent	✓ Excellent

Native library ecosystem	Legacy	Excellent	Excellent and growing
Memory-intensive applications	✗ Poor	✓ Excellent	✓ Excellent
Modern hardware availability	Low	Very high	Very high
Recommended for new systems	✗ No	✓ Yes	✓ Yes

14. Migration strategy

If you currently run Java on 32-bit x86, a practical migration path is:

Step	Action
1	Identify all x86_32 production systems
2	Identify the operating system architecture
3	Identify the JDK architecture
4	Inventory JNI/native libraries
5	Check vendor dependencies
6	Provision 64-bit infrastructure
7	Install a 64-bit JDK
8	Rebuild native libraries for 64-bit
9	Run application integration tests
10	Run performance/load tests
11	Validate containers and deployment scripts
12	Move production workloads
13	Retire the 32-bit environment

Oracle recommends a migration process that includes running the application on the newer JDK, updating third-party libraries, recompiling where needed, and using tools such as `jdeps` during migration. ([Oracle Docs](#))

15. Performance implications

Area	Expected effect
Java source execution	Usually no direct change
JVM maintenance	Less architecture-specific complexity
Memory capacity	64-bit is much better suited to large-memory systems
Modern JIT	Better alignment with current JVM development
Native libraries	May require recompilation
Application performance	Must be benchmarked after migration
Startup time	Not inherently improved simply because <code>x86_32</code> is deprecated
Throughput	Depends on workload and hardware
GC performance	Depends on JVM, heap and workload
Cloud performance	Generally much more practical on modern 64-bit architectures

Do not interpret JEP 501 as a guaranteed performance improvement. Its main benefit is reducing maintenance and allowing JVM development to focus on modern platforms.

16. JEP 501 vs JEP 479 vs JEP 503

This distinction is important because these JEPs are easy to confuse.

JEP	Java version	What happened
JEP 449	Java 21	Deprecated Windows 32-bit x86 for removal

JEP 479	Java 24	Removed Windows 32-bit x86 port
JEP 501	Java 24	Deprecated the remaining 32-bit x86 port , effectively Linux x86_32, for removal
JEP 503	Java 25	Removed the 32-bit x86 port

So the user's feature is specifically:

JEP 501 = the deprecation stage in Java 24.

By Java 25, JEP 503 completed the removal. ([OpenJDK Bug System](#))

17. When to choose which architecture?

Requirement	Recommended choice
Traditional enterprise server	x86-64
AWS/Azure/GCP cloud	x86-64 or ARM64
Kubernetes	x86-64 or ARM64
Large-memory application	x86-64 / ARM64
High-throughput Java server	x86-64 / ARM64
Modern embedded system	ARM64 or x86-64
Existing 32-bit legacy application	Plan migration
Old 32-bit industrial equipment	Evaluate hardware replacement
New application	Never choose x86_32 as the strategic target

18. Overall decision table

Question	Answer
Is JEP 501 a Java API?	✗ No
Does it require changing normal Java code?	Usually ✗ No
Does it affect Java platform support?	✓ Yes
Does it affect 64-bit x86?	✗ No
Does it affect ARM64?	✗ No
Does it affect 32-bit x86?	✓ Yes
Should new applications target x86_32?	✗ No
Should existing x86_32 systems be migrated?	✓ Yes
Is x86-64 a good replacement?	✓ Yes
Is ARM64 a good replacement?	✓ Yes
Is Zero a possible fallback for 32-bit x86?	✓ Technically, but specialized and not a normal performance replacement
Is this mainly a performance feature?	✗ No
Is this mainly a platform-maintenance change?	✓ Yes
Was the port eventually removed?	✓ Yes, in Java 25 via JEP 503

Bottom line

Recommendation	Verdict
New Java application	🔍 Use 64-bit x86-64 or ARM64
Existing 32-bit Java application	🔍 Start migration planning
32-bit Linux production system	🔍 Treat as legacy infrastructure
32-bit JNI/native application	🔍 Audit and rebuild native dependencies

New enterprise infrastructure	☒ Avoid x86_32 completely
Long-lived product	☒ Target 64-bit platforms
Need to keep an old 32-bit machine alive	☒ Possible only as a legacy strategy; don't build new dependencies around it

The key idea: JEP 501 is less about changing your Java code and more about **moving your Java runtime infrastructure from obsolete 32-bit x86 to modern 64-bit platforms**. The deprecation in Java 24 was the warning; **Java 25's JEP 503 completed the removal**. ([OpenJDK Mail](#))